MILESTONE 1 · ANALYTICAL REPORT

Formula 1 Podium Prediction

For every driver on the starting grid of a race, predict whether they finish in the top three, using only information known after qualifying and before the start.

Team 10	Omar Ahmed Fawzy Ahmed Amin · Khaled Mohamed Karam Alah Salleh · Youssef Ahmed Fathy Elsayed Elfeky · Muhamed Muhamed Muharam Mustafa

Kaggle notebook	kaggle.com/code/muhamedmuharam/notebook699276446c
GitHub repository	github.com/MuhamedMuharam/ACL-Team-10
Dataset	Formula 1 World Championship (1950–2024), Kaggle / Ergast, 14 relational tables
Notebook	`F1_Podium_Prediction.ipynb`: one Run All, sections 1–13; every decision (D1–D30) is logged in `DECISIONS.md`

Summary

- **Data.** 26,759 raw result rows become **24,715 driver-race rows**: the cars on the grid at the prediction cutoff, one row per driver per race, with the Indianapolis 500 and failed qualifiers removed. Models are trained on **every season from 1950 to 2019**, tuned on 2020–2021 and tested once on 2022–2024. A later start was tested: none predicts better, and starting in 1994 is significantly worse.
- **Q1, front-row conversion.** Front-row cars podiumed more often after 2014: **68.0% → 75.9%** ($p = 0.013$). Suzuka, Yas Marina and Monaco convert best after 2014, but circuits are not statistically distinguishable in either period, so a real change in the *ranking* cannot be shown. A front-row qualifier moved back by a penalty podiums only 28%, against 73% for one who starts there.
- **Q2, home advantage.** Comparing drivers who started from similar grid slots in the same era, home drivers podium **0.8 percentage points less** often, and the logistic odds ratio is 0.93 (95% CI 0.79–1.11). There is **no evidence of a home advantage**.
- **Q3, mechanical retirements** (teams grouped by lineage). They fell from **8.9% to 5.5% of race starts**. Highest in 2014–2021: Caterham (21.2%), Toro Rosso → AlphaTauri → RB (14.4%), Lotus F1 → Renault → Alpine (13.2%). In 2022–2024 the Enstone team (Alpine, 10.4%) is highest, but team differences are not significant.
- **Models.** Five learned models on 18 features reach validation PR-AUC 0.72–0.74, against 0.64 for a grid-only rule and 0.15 for a random model. None beats the **logistic-regression baseline** by more than noise, so it is the recommended model: **test PR-AUC 0.722** (tied best), ROC-AUC 0.937, well calibrated. All five are explained and have their own inference function.
- **Explainability.** In all five models, starting position dominates, followed by season-long strength (standings) and recent form. These explanations describe what the models rely on, **not what causes a podium**.

1. Data, audit and cleaning

1.1 Dataset, unit of analysis and the prediction cutoff

The data is spread over 14 linked tables. One row of the modeling table is **one driver entering one race**, and the target is `podium = positionOrder ≤ 3`. The prediction is made **after qualifying and before the start**. Anything produced by the race itself is leakage: finishing position, points, laps, status, pit stops, lap times, and the standings published after that race.

table	rows	use for race R	table	rows	use for race R
lap_times	589,081	post-race (not used)	races	1,125	pre-race (year, round, date, circuit)
driver_standings	34,863	previous race only	drivers	861	pre-race (date of birth, nationality)
results	26,759	grid only; the rest is the target or post-race	sprint_results	360	pre-race, but not used (D12)
constructor_standings	13,391	previous race only	constructors	212	lookup
constructor_results	12,625	not used	status	139	post-race (Q3 only)
pit_stops	11,371	post-race (not used)	circuits	77	pre-race (country)
qualifying	10,494	pre-race (position, Q1 time)	seasons	75	lookup

Table 1.1 The 14 tables and whether the current race's values may be used as features.

All files are first read **as plain text**, so the audit sees them exactly as written, with no type guessing and no silent NaN conversion. The raw files are never modified: an MD5 fingerprint of each file is taken at the start and checked after cleaning, and all 14 are byte-for-byte unchanged (D1). An assertion also checks the dataset version (last season 2024).

1.2 Audit of the raw data (before cleaning)

Sentinel values. Missing values are written as the literal text `\N`. Pandas reading the text sees **0 missing values**, so every one of them is hidden. Placeholders appear in 30 columns of 6 tables, and 68 cells are empty strings (all in `qualifying.q2/q3`), which mean the same thing. The missing values are *structural*:

- session dates exist only for recent seasons;
- finishing times are missing for non-finishers;
- `q2` / `q3` are missing for drivers knocked out earlier.

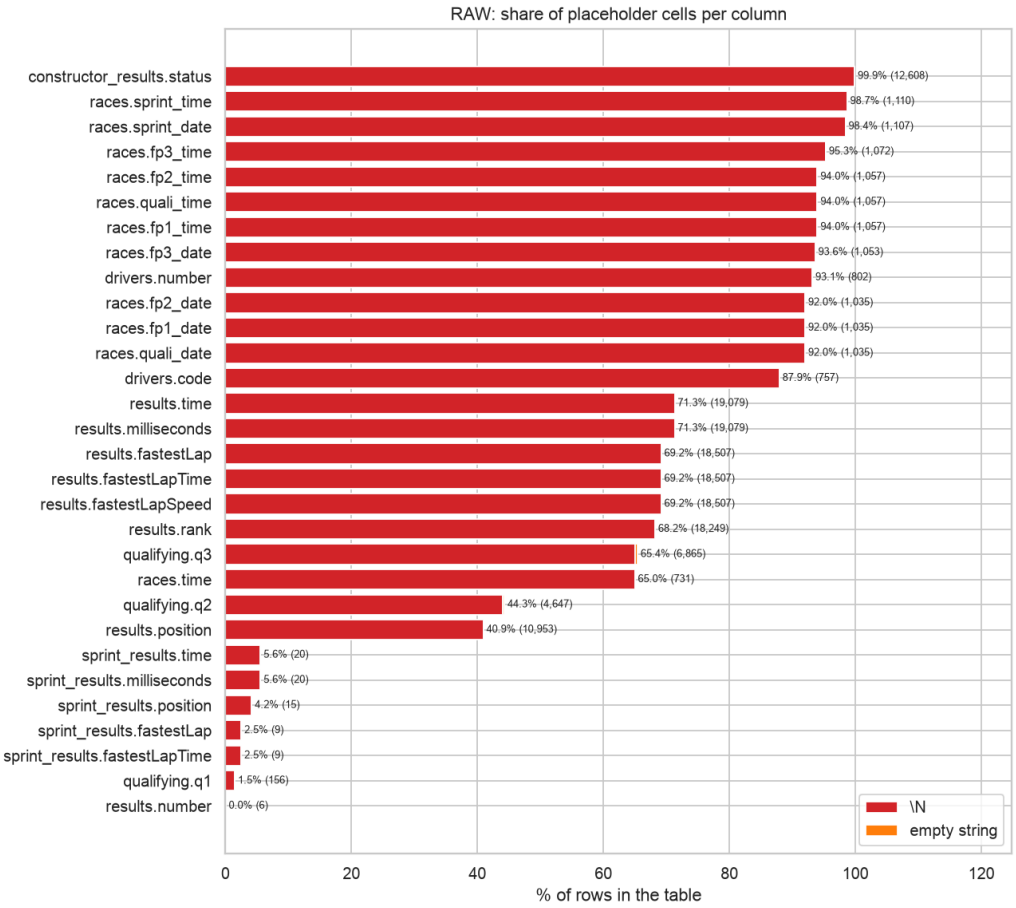


Figure 1.1 Raw data: share of placeholder cells (`\N` red, empty string orange) per column.

How the data changes over 75 seasons. Early races had far more entries than today's 20 cars: up to almost 40 rows per race around 1989, with failed qualifiers included. As a result, only about 8–12% of rows are podiums in the busy eras, against about 15% today. A raw `grid = 0` means two different things (Table 1.2): before 1995 it is almost always a driver who never got onto the grid, while from 1995 it is mostly a pit-lane start.

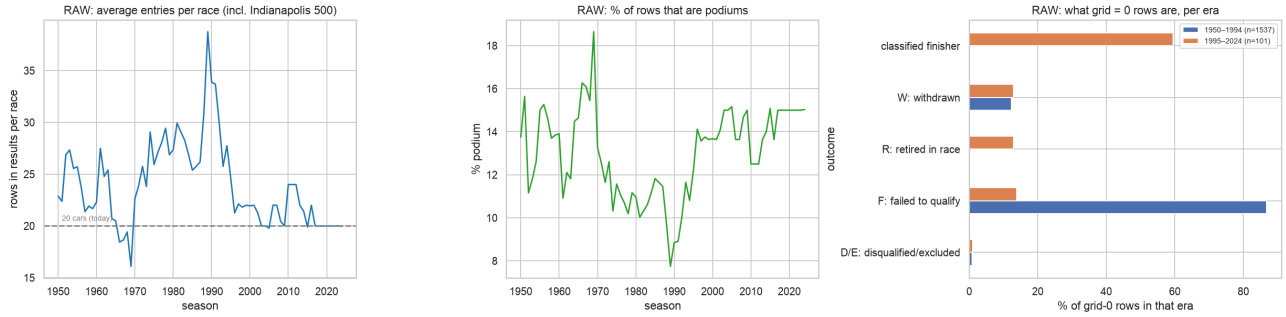


Figure 1.2 Raw data: (a) average entries per race, (b) share of rows that are podiums, (c) what `grid = 0` rows are, per era.

outcome of a raw grid-0 row	1950–1994	1995–2024
F: failed to qualify	1,334	14
W: withdrawn	190	13
D/E: disqualified or excluded	12	1
R: retired in the race	1	13
classified finisher	0	60

Table 1.2 The 1,638 raw rows with `grid = 0`. A grid 0 is a pit-lane start only for drivers who actually raced, which decides D6 and D7.

Qualifying coverage drift. `qualifying.csv` is empty before 1994. It covers 94% of races in 1994 and all of 1995, only 6–59% of races in 1996–2002, and every race from 2003. From 2003, at least 98.9% of result rows in every season have a qualifying row. So qualifying features are missing *by design* for most rows before 2003, and for those seasons the grid is the only record of the starting order.

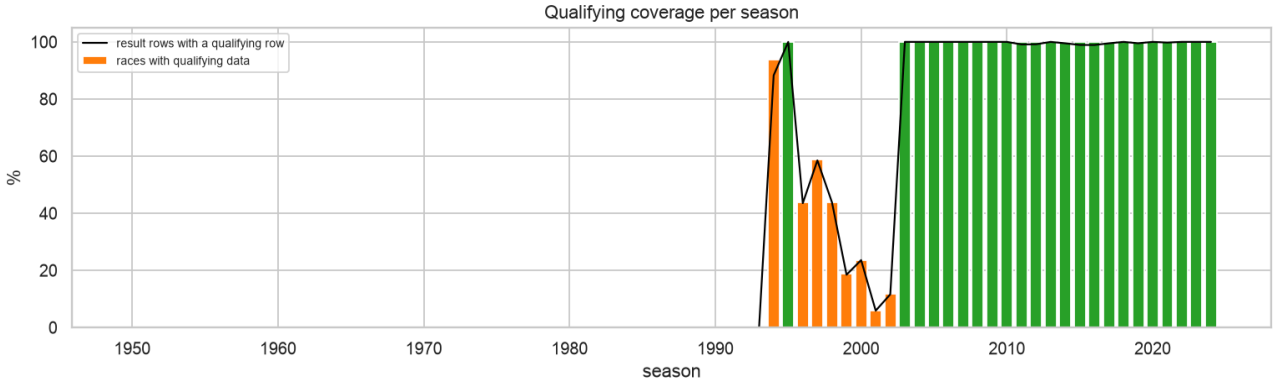


Figure 1.3 Share of races with qualifying data (bars; green = complete) and share of result rows with a qualifying row (line), per season.

Inconsistent text values were found by code checks, not by eye:

- the nationality `"Argentinian "` has a trailing space, and is the same nationality as `"Argentine"`;
- the circuit country appears both as `"United States"` and as `"USA"`;
- one constructor name contains an HTML code (`"Lotus–Pratt & Whitney"`).

Left alone, the first two would split one country into two groups in Q2. Dual or historical nationalities ("American-Italian", "East German") are real values, not errors. **Dates and durations stored as text** were also listed: dates; lap times such as `1:27.452`; the results `time` column, which mixes the winner's total time with gaps such as `+5.478`; pit-stop durations; and times of day.

1.3 Cleaning

step	what was done	result / check	decision
Sentinels → missing	Every file re-read with `\N` and `""` as missing; `keep_default_na=False` so real text such as "NA" is kept	Cell-by-cell check: NaN ⇔ placeholder; 0 cells disagree (asserted)	—
Semantic typing	Dates → datetime; lap, qualifying and fastest-lap times → seconds; pit durations → seconds; race times from `milliseconds`; `rank = 0` → missing	8 conversions, 0 parse failures (asserted); parsed lap times match the milliseconds column	—
Plausibility rule	A race time below laps × 30 s is impossible	18 rows (2024 Belgian GP, where the column holds gaps) set to missing	—
Text normalisation	Trailing spaces, "Argentinian" → "Argentine", "United States" → "USA", HTML codes unescaped	Nationalities 43 → 42, countries 35 → 34	D2
Primary and foreign keys	14 primary keys, 5 natural join keys, 26 foreign-key links (incl. lap / pit / qualifying rows → a result in that race)	All unique and non-null; 0 orphans in all 26 links; every race has results	—

Status categories	Each of the 139 labels mapped explicitly to one of 8 categories (Table 1.4)	Asserted: every label mapped, none twice	D3
Grain: shared drives	85 (raceId, driverId) pairs (176 rows, 1950–1978); three keep rules compared (Table 1.5)	26,759 → 26,668 rows; (raceId, driverId) unique (asserted)	D4
Indianapolis 500	Removed (1950–1960): a separate competition	–381 rows	D5
Cars on the grid at the cutoff	Removed failed qualifiers and entries never on the grid; kept cars that qualified but did not start	–1,365 and –207 rows → 24,715 rows in 1,114 races	D6
Starting position	Grid re-ranked 1 … field size among the kept cars; pit-lane starters to the last slot with a flag	73 pit-lane starts; 1 ≤ slot ≤ field size (asserted)	D7
Target	`podium = positionOrder ≤ 3`; `positionOrder` never used as a feature	Podium rate 13.5%; 1,113 races with 3 podium rows, 1 with 4	D8

Table 1.3 Cleaning steps in order, each with its count and the decision that justifies it (decisions in `DECISIONS.md`).

Status categories (D3). The brief defines mechanical as "a component of the car failed", which excludes crashes, the driver and regulation decisions. A label is mechanical only if it names a failed component. Damage usually caused by contact or the pit crew (tyres, wheels, wings, debris) is `ambiguous`, so those labels are treated the same way. "Withdrew" is decided per row in D6.

category	labels	result rows	examples
finished	35	15,333	Finished, +1 Lap, Not classified
mechanical	65	6,317	Engine, Gearbox, Hydraulics, Brakes, Power Unit
accident_driver	11	2,839	Accident, Collision, Spun off, Illness
did_not_qualify	3	1,365	Did not qualify, Did not prequalify, 107% Rule
ambiguous	18	489	Retired, Puncture, Wheel, Front wing, Debris
withdrawn	1	248	Withdrew
regulation	3	156	Disqualified, Excluded, Underweight
team_operational	3	12	Wheel nut, Fuel rig, Refuelling

Table 1.4 The 139 status labels in 8 categories, with how often each occurs in `results`.

Duplicate drivers in one race (D4). In the 1950s a driver could take over a team-mate's car mid-race, and then has two rows in the same race. At the 1957 British GP, Moss started car #18 from pole and later won in car #20, while Brooks drove #18 until its engine failed, so both drivers appear twice. Three keep rules were compared on two checks a clean table must pass:

rule	races with > 3 podium rows	drivers sharing one car
A: keep the best `positionOrder`	15	121
B: keep the driver's own entry (lowest `resultId`)	2	36
C: keep the car only he is recorded in, else B (chosen)	**1**	**30**

Table 1.5 Duplicate-rule comparison (lower is cleaner). All rules remove the same 67 non-Indy rows; they differ in which row survives.

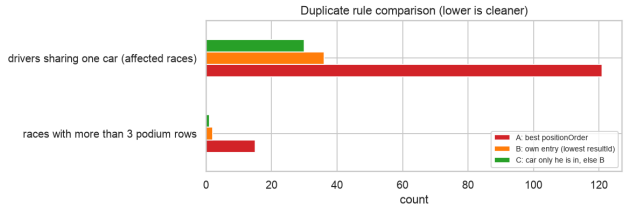


Figure 1.4 The three duplicate rules on the two integrity checks.

Rule A credits a shared car's result to both drivers (Moss and Brooks "won" in 1957). It is also outcome-based row selection, which is a form of target leakage. Rule C keeps the car the driver started in, which is the car his pre-race information describes. For example, Ascari in 1951 started 3rd and retired, instead of "started 6th, finished 2nd" in his team-mate's car.

All affected rows are from 1950–1978, **inside the training window** (D17). Rules A and B differ in 15 podium labels, B and C in one (Ascari, 1951), so C is the rule that keeps those training labels right.

The Indianapolis 500 (D5). It counted for the championship, but of its 107 drivers only 4 ever started another championship race. Its entry lists average 35 per race (55 in 1954), against 22 at other races of that decade. Of its 381 rows, 380 are American drivers racing in the USA, more than three times all other American home starts in 75 seasons. Keeping it would distort field sizes, podium rates and the home-race question; since the model trains from 1950, it would also put 381 oval-race rows into training.

Who is in the table (D6). The prediction is made after qualifying. At that moment, failed qualifiers (1,365 rows) and entries that never reached the grid (207 rows) are already out. Cars that qualified but then failed to start are still on the grid and are kept as non-podiums (137 rows, e.g. Leclerc on pole at Monaco 2021): dropping them would use information from after the cutoff.

Starting position (D7). Left as 0, a pit-lane start would look better than pole. In old seasons the raw grid numbers also have holes and ties (1981 German GP: no slots 19, 20, 24, 25, 27, and the last of 24 starters on grid 28). The grid is therefore re-ranked among the cars on the grid, and pit-lane starters go to the last slot with their own flag.

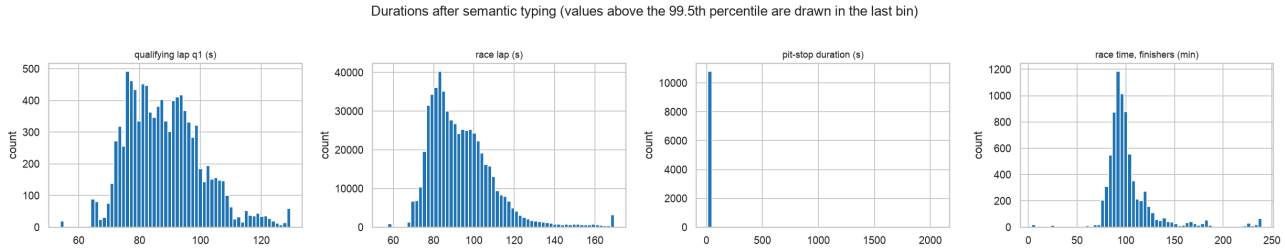


Figure 1.5 Durations after semantic typing. The long tails are real events: 708 laps over 5 minutes (red flags, safety cars) and 517 "pit stops" over a minute (red-flag stoppages). They are not features, so they are kept as they are.

1.4 EDA after cleaning: what the cleaning changed

	rows	duplicate (race, driver) rows	grid = 0 rows	slot > number of cars	podium rate	races
raw	26,759	91	1,638	13	12.69%	1,125
cleaned	24,715	0	0	0	13.53%	1,114

Table 1.6 The modeling grain before and after cleaning.

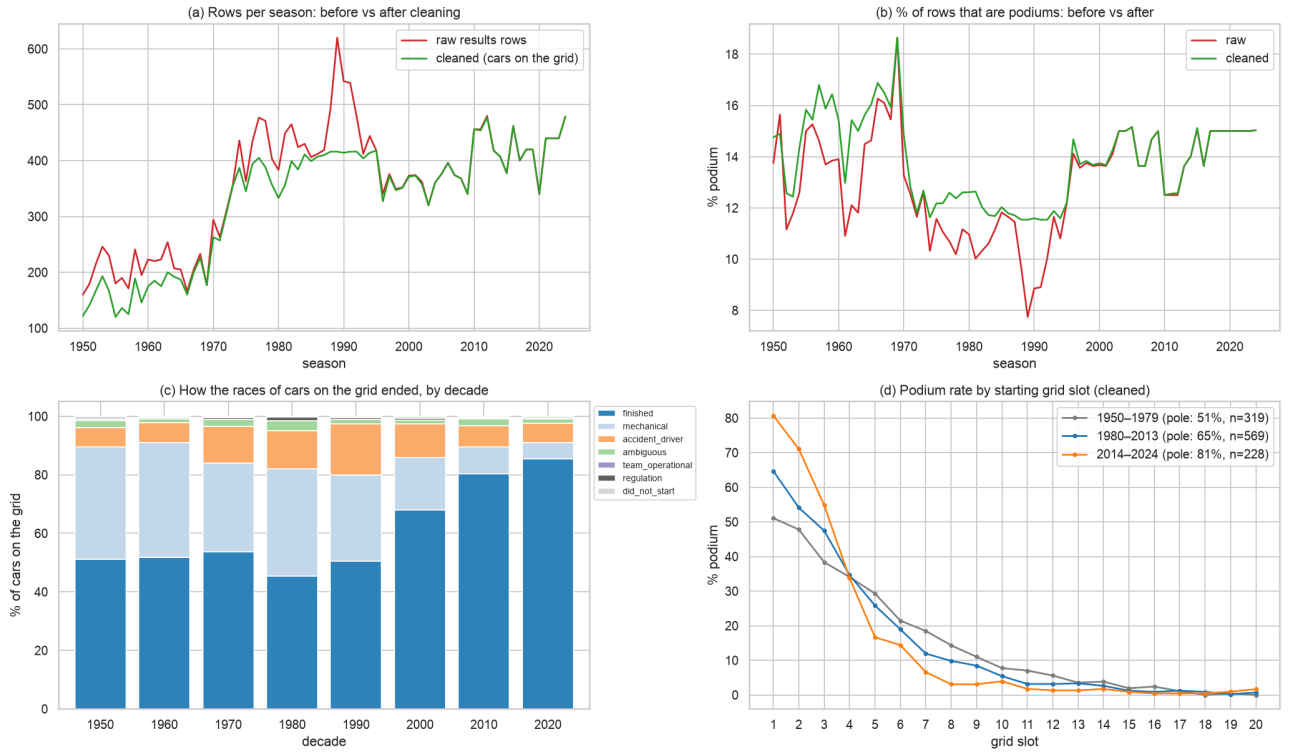


Figure 1.6 (a) Rows per season and (b) podium base rate, before vs after cleaning. (c) How the races of cars on the grid ended, by decade. (d) Podium rate by starting slot in three eras.

- **(a, b)** Cleaning removes rows mostly from the 1950s and the busy late-1970s to 1990s, where entry lists were far larger than the grid. Removing failed qualifiers, who can never podium, raises the base rate in those eras (late 1980s: from about 8–10% to about 12%). Modern seasons are almost untouched (about 15%).
- **(c) Reliability changed enormously.** In the 1950s–1980s around a third of the cars on the grid retired with a mechanical failure; in the 2020s most cars finish.
- **(d) Starting position predicts podiums in every era, and much more strongly today.** Pole converts 51% in 1950–1979, 65% in 1980–2013 and 81% in 2014–2024. This motivates the grid features, and it is why the training window was tested rather than assumed (Section 3.4).

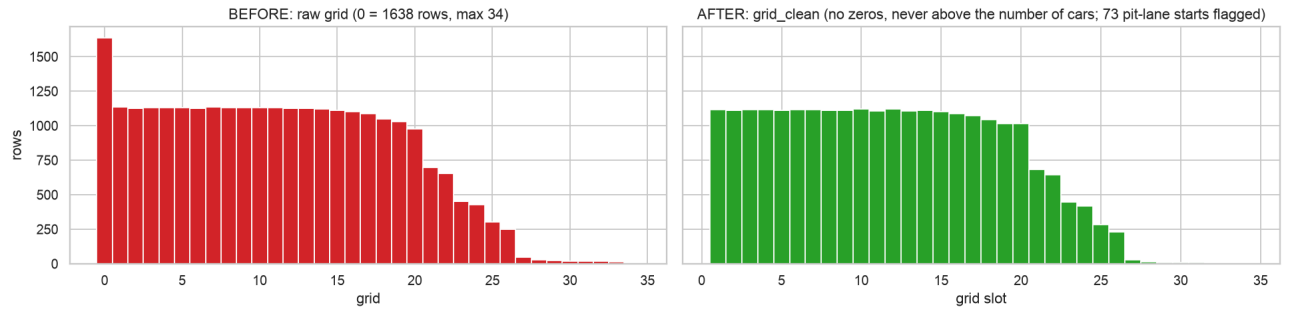


Figure 1.7 Starting position before (raw `grid`, with a spike of 1,638 zeros) and after the re-ranking (no zeros, never behind the last car).

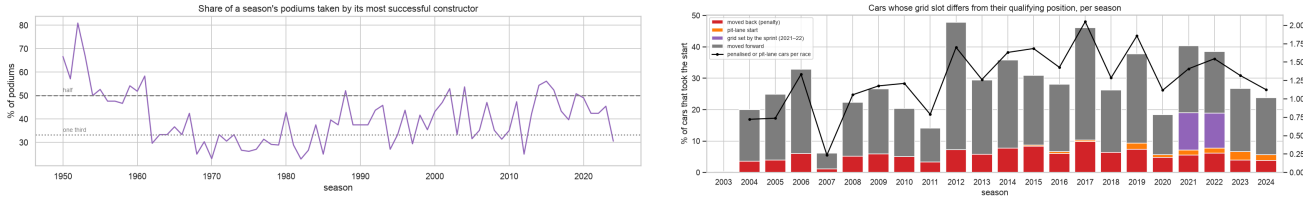


Figure 1.8 Left: the share of a season's podiums taken by its most successful constructor. The median is 37.5%, and above half in 15 seasons (e.g. 2014–2016, 2019). The car matters, so constructor features are included. Right: cars whose grid slot differs from their qualifying position. Penalised or pit-lane cars per race rose from 1.0 in 2003–2013 to 1.5 in 2014–2024. On 2021–22 sprint weekends the sprint set the grid. Grid and qualifying are therefore both kept, with their difference as a feature.

2. Data-engineering questions

Each answer joins several tables, shows a plot, and states its grain, how missing values were handled, and whether the pre-race cutoff matters. All three use the cleaned driver-race table of Section 1 (starters only, no Indianapolis 500, one row per driver per race).

2.1 Q1. Which circuits convert a front-row start into a podium most often, and does the ranking change after 2014?

Joins: `results` ⋈ `races` ⋈ `circuits` ⋈ `qualifying` ⋈ `sprint_results`. Set-up (D9):

- The front row is measured two ways: by **grid** (slot 1–2 after penalties) and by **qualifying** (1st–2nd).
- The periods are **2003–2013 vs 2014–2024**. 2003 is the first season with complete qualifying data, so both definitions use the same rows.
- A circuit is ranked only with at least 8 front-row starts (4 races) in both periods.
- Every rate gets a 95% Wilson interval. Chi-square tests ask whether circuits differ at all and whether the overall rate changed; Spearman's rank correlation asks whether the order carries over.

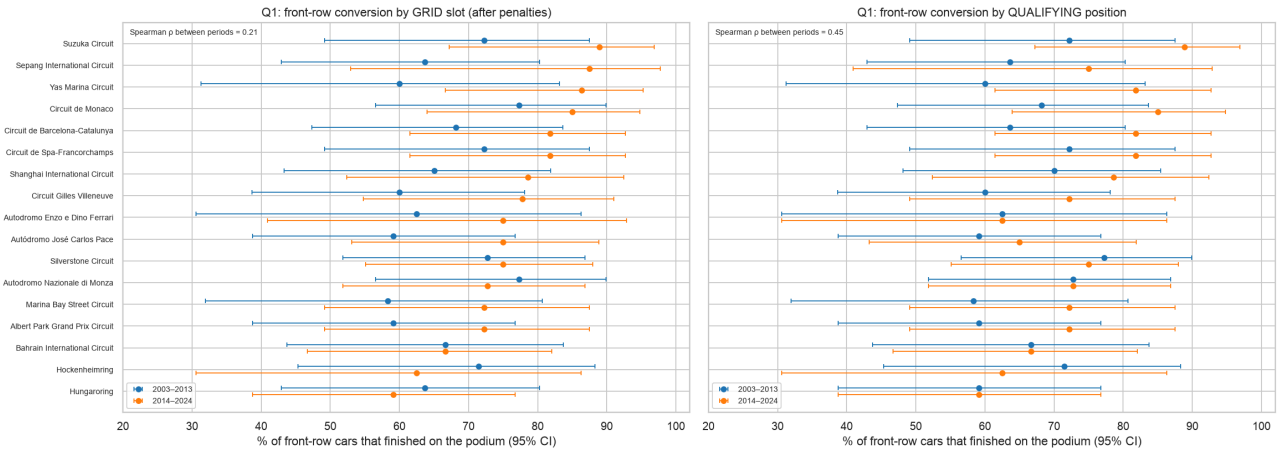


Figure 2.1 Front-row conversion per circuit with 95% intervals, 2003–2013 (blue) vs 2014–2024 (orange). Left: by grid slot. Right: by qualifying position (same circuit order).

circuit (grid definition)	2003–13 rate	rank	n	2014–24 rate	rank	n
Suzuka	72%	4	18	89%	1	18
Sepang	64%	10	22	88%	2	8
Yas Marina	60%	13	10	86%	3	22
Monaco	77%	1	22	85%	4	20
Spa-Francorchamps	72%	4	18	82%	5	22
Silverstone	73%	3	22	75%	9	24
Monza	77%	1	22	73%	12	22
Hungaroring	64%	10	22	59%	17	22

Table 2.1 Selected circuits from the full 17-circuit ranking (notebook §5.1): conversion rate, rank and number of front-row starts per period.

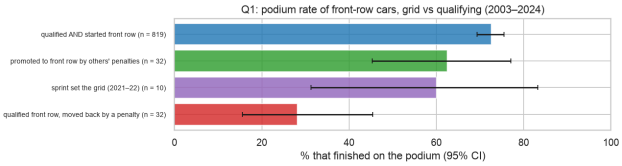


Figure 2.2 Podium rate of front-row cars by how their grid slot relates to their qualifying position (95% intervals).

front-row case, 2003–2024	n	podium
qualified and started on the front row	819	72.5%
promoted by others' penalties	32	62.5%
sprint set the grid (2021–22)	10	60.0%
qualified front row, moved back by a penalty	32	28.1%

Table 2.2 Grid vs qualifying: why both definitions matter.

Answer.

- **Front-row starts convert more often after 2014:** 68.0% (n = 400) → 75.9% (n = 456), chi-square p = 0.013.
- **Best converters.** After 2014: Suzuka (89%), Sepang (88%, only 8 starts), Yas Marina (86%) and Monaco (85%). Before: Monaco and Monza (77%) and Silverstone (73%).
- **The ranking appears to change, but the change cannot be shown.** Monza falls from joint 1st to 12th and Yas Marina rises from 13th to 3rd. But each circuit has only 8–24 starts per period, the circuits are not statistically distinguishable in either period (p = 0.99 and 0.80), and the rank correlation between periods is weak and not significant (ρ = 0.21, p = 0.42). Monza's fall is one podium fewer (17 → 16 of 22), because all circuits sit between 58% and 89%. By qualifying position ρ = 0.45 (p = 0.07). The robust finding is the overall rise.
- **Grid vs qualifying matters.** A front-row qualifier moved back by a penalty podiums only 28%, against 73% for one who starts there (Table 2.2). The grid after penalties is therefore the better measure of the real starting chance.

Grain / missingness / cutoff. One row per driver-race, aggregated per circuit and period. The 22 rows from 2003–2024 without a qualifying position all started 19th or lower, so none is a front-row case; they are kept, not imputed. Grid, qualifying and the sprint result are all known before the race.

2.2 Q2. Do drivers podium more often at home, after controlling for starting position?

Joins: `results` ⋈ `races` ⋈ `circuits` (country) ⋈ `drivers` (nationality). Set-up (D10):

- A home race means the circuit's country matches the driver's nationality. The match uses an explicit hand-written map (e.g. "British" → "UK"), and both lists of values were checked for gaps. The circuit's location decides, not the race name, and a dual nationality counts both countries.
- **Control:** home and away are compared within the same grid bucket (1 / 2–3 / 4–6 / 7–10 / 11+) × era (1950–79 / 1980–2013 / 2014–24). This is a pivot table, drawn as a heatmap; the cells are averaged with weights = home starts.
- **Test:** a logistic regression `podium ~ home + grid slot + era` with race-clustered standard errors (drivers in one race are not independent).

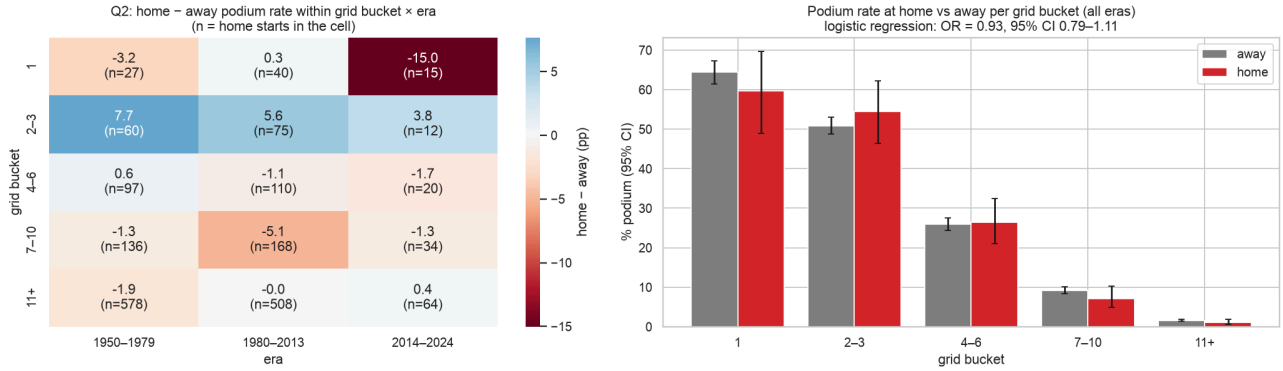


Figure 2.3 Left: home − away podium rate (percentage points) within each grid bucket × era; n = home starts. Right: podium rate at home (red) vs away (grey) per grid bucket, all eras, with 95% intervals.

estimate	home − away (pp)
raw, no control	−2.12
within grid bucket	−0.60
within grid bucket × era (main)	**−0.83**

Table 2.3 Home effect in percentage points (1,944 home starts). Logistic regression (clustered by race): odds ratio 0.93 (95% CI 0.79–1.11), p = 0.43.

Answer.

- **No evidence that drivers podium more often at home once starting position is accounted for.** The odds ratio 0.93 has an interval (0.79–1.11) that includes 1.
- **The raw gap comes from who races at home.** Raw, home drivers podium 2.1 pp less often. 13.1% of home starts are by drivers with five or fewer career starts (one-off local entrants), against 1.4% of away starts.
- **Controlling shrinks the gap** to −0.6 pp within grid buckets and −0.8 pp within grid bucket × era, about 61% less than raw. The 15 heatmap cells range from −15.0 to +7.7 pp, with signs in both directions in every era: no consistent home effect.
- This is an association, not a causal effect.

Grain / missingness / cutoff. One row per driver-race. Nationality and circuit country have no missing values; nationalities with no circuit in their country (e.g. Finnish) are always "away". Nationality, circuit and grid are known before the race, so `home_race` would be a legal feature; it was built as a candidate, but it shows no signal and is not a model input (D30).

2.3 Q3. Which constructors have the highest mechanical-retirement rate in 2014–2021 vs 2022–2024?

Joins: `results` ⋈ `races` ⋈ `status` (category, D3) ⋈ `constructors`. Set-up (D11):

- Rate = mechanical retirements ÷ race starts. Cars that were on the grid but did not take the start are not starts.
- The strict rate counts only labels that name a failed component. The broad rate adds ambiguous labels, as a robustness check.
- **Teams are grouped by lineage.** A renamed team keeps its factory, staff and car, but gets a new `constructorId`, and four renames fall on the period boundary (Racing Point → Aston Martin and

Renault → Alpine in 2021, AlphaTauri → RB and Alfa Romeo → Sauber in 2024). A small dictionary joins Toro Rosso → AlphaTauri → RB, Force India → Racing Point → Aston Martin, Lotus F1 → Renault → Alpine, Sauber ↔ Alfa Romeo and Marussia → Manor. Each lineage is then compared with itself across the two periods.

- Only lineages with ≥ 30 starts are ranked; rates have 95% Wilson intervals, and a chi-square test asks whether teams differ at all.

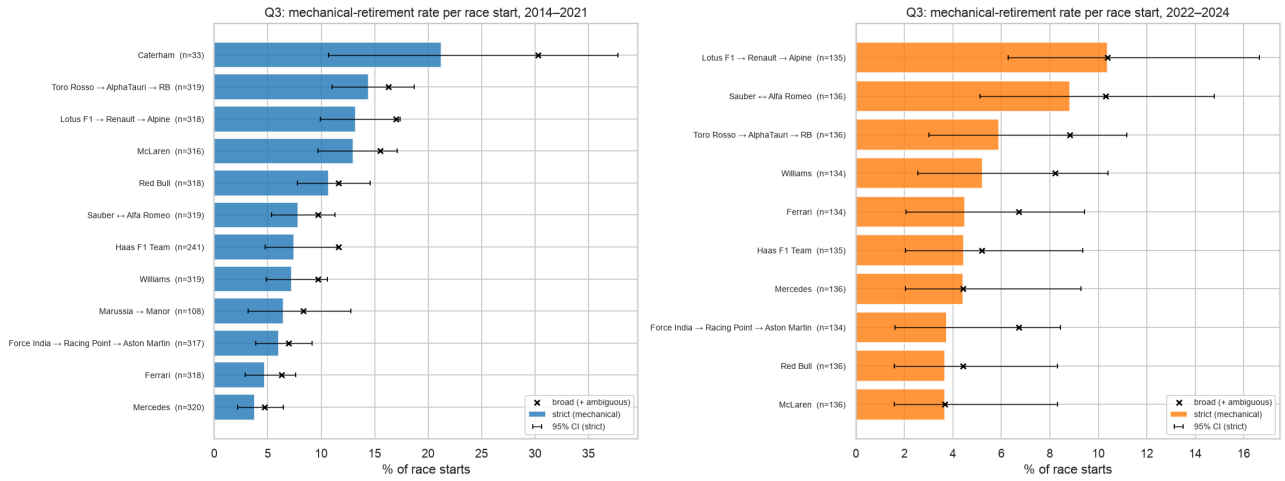


Figure 2.4 Mechanical-retirement rate per race start, by team lineage: bars = strict rate with 95% interval, × = broad rate (+ ambiguous labels), n = starts.

2014–2021 (overall 8.9%)	starts	strict	95% CI	2022–2024 (overall 5.5%)	starts	strict	95% CI
Caterham	33	21.2%	10.7–37.8	Lotus F1 → Renault → Alpine	135	10.4%	6.3–16.7
Toro Rosso → AlphaTauri → RB	319	14.4%	11.0–18.7	Sauber ↔ Alfa Romeo	136	8.8%	5.1–14.8
Lotus F1 → Renault → Alpine	318	13.2%	9.9–17.4	Toro Rosso → AlphaTauri → RB	136	5.9%	3.0–11.2
McLaren	316	13.0%	9.7–17.1	Williams	134	5.2%	2.6–10.4
…				…			
Force India → Racing Point → Aston Martin	317	6.0%	3.9–9.2	McLaren	136	3.7%	1.6–8.3
Ferrari	318	4.7%	2.9–7.6	Red Bull	136	3.7%	1.6–8.3
Mercedes	320	3.8%	2.2–6.4	Force India → Racing Point → Aston Martin	134	3.7%	1.6–8.4

Table 2.4 Highest and lowest strict rates per period (full tables with broad rates: notebook §5.3).

Answer.

- **2014–2021.** Caterham (21.2%, only 33 starts), Toro Rosso → AlphaTauri → RB (14.4%), Lotus F1 → Renault → Alpine (13.2%) and McLaren (13.0%) had the highest mechanical-retirement rates. Mercedes, the dominant team, was the lowest (3.8%). Teams clearly differ ($p < 0.001$).
- **2022–2024.** Lotus F1 → Renault → Alpine (10.4%) and Sauber ↔ Alfa Romeo (8.8%) are highest, but differences between teams are **not significant** ($p = 0.18$) and the intervals overlap, so this order is weak evidence.
- **Change between the periods.** Failures became rarer overall (8.9% → 5.5%) and for almost every lineage. McLaren (13.0% → 3.7%), Red Bull (10.7% → 3.7%) and Toro Rosso → AlphaTauri → RB (14.4% → 5.9%) improved most; only Sauber ↔ Alfa Romeo (7.8% → 8.8%) and Mercedes (3.8% → 4.4%) rose slightly. For the 10 lineages present in both periods the rank correlation is +0.27 ($p = 0.45$): no evidence that the same teams stay the least reliable.
- **Why lineage matters.** Grouped by `constructorId`, "Alpine 2014–2021" was a single season and "RB 2022–2024" only 2024, and the cross-period correlation came out −0.49 because identities were mixed.
- **Robustness.** The broad rate keeps the same top four in 2014–2021 and the same top three in 2022–2024.

Grain / missingness / cutoff. One row per car that took the start, aggregated per lineage and period; status has no missing values. Status is post-race information: it is used only in this analysis, never as a feature for the current race. A team's past reliability is a legal feature when computed from earlier races only (`con_mech_rate_last10`).

3. Features, split and preprocessing

3.1 Selected features, and why each is valid at the pre-race cutoff

We built **21 candidate features** in five groups and evaluated each in the EDA (Section 3.3). The models use the **18** that show a signal; the three with none (`pit_lane_start`, `home_race`, `field_size`) are not model inputs (D30), and Section 4.6 checks that adding them back never helps. The groups are reused by the ablation. Every feature is known after qualifying and before the start.

group	feature	definition	why it is known before the start
grid	`grid_pos`	starting slot after penalties, re-ranked among the cars on the grid (pit-lane start = last slot, D7)	the starting grid is fixed after qualifying and penalties
	`grid_pct`	(slot − 1) ÷ (field size − 1): 0 = pole, 1 = last	
qualifying	`quali_position`	qualifying classification	qualifying runs before the race; on 2021–22 sprint weekends the sprint, also before the race, sets the reference
	`quali_gap_pct`	the driver's Q1 lap vs the fastest Q1 lap of that race, in % (laps > 20% off are not real attempts and become missing: 10 of 10,338)	
	`grid_penalty`	grid slot − qualifying position (on 2021–22 sprint weekends: − sprint result)	
standings	`drv_pts_share_before`, `con_pts_share_before`	championship points ÷ the leader's points	taken from the table published after the previous race of the season (season opener: the previous season's final table), never from the current race
	`drv_rank_before`, `con_rank_before`	championship position	
	`drv_win_share_before`, `con_win_share_before`	wins ÷ races run so far in the season	
form	`drv_podium_rate_last5`	share of the driver's previous 5 races on the podium	rolling windows over earlier races only: `shift(1)` before `rolling()`, in date order; the team is collapsed to one row per race first, so a team-mate's current result never leaks in
	`drv_avg_finish_last5`	average finishing position as a fraction of the field, previous 5 races	
	`con_podium_rate_last5`	same as the driver version, for the team (both cars)	
	`con_avg_finish_last5`	same as the driver version, for the team (both cars)	
	`con_mech_rate_last10`	share of the team's cars with a mechanical failure, previous 10 races	

context	`driver_age`	age at the race date	date of birth and the number of earlier races are known beforehand
	`experience`	number of previous races on the grid	

Table 3.1 The 18 model features (D13–D16, D30). Shares and ranks are used instead of raw points because points systems (8, 9, 10, 25 for a win) and season lengths (7–24 races) changed. Raw lap times are not used, because they partly measure track length (Monaco ≈ 70 s, Spa ≈ 105 s); a gap inside one race normalises circuit, season and weather at once, like the FIA's 107% rule (D15). Same-weekend sprint results are not used, because sprints are absent from every training season (D12). `field_size` remains an input of the inference function, because `grid_pct` is computed from it.

3.2 Leakage controls

- **Standings from before the race.** At the 2023 Belgian GP, Verstappen's standings row says 314 points, which already includes that race. The feature uses the 281 he had before it. An assertion checks that every `prev_raceId` lies strictly earlier in time than its race.
- **Form from past races only.** `shift(1)` keeps the current race out of its own window, and races are ordered by date, not by `raceId`. A hand check recomputes Verstappen's podium rate over the 5 races before race 989 from the raw rows (0.40 = feature, asserted).
- **No post-race column in the features.** An assertion checks every candidate feature against all post-race columns: `positionOrder`, points, laps, status, finishing rank, mechanical flag, and the rest.
- **Choices on training data only.** Feature evidence, the 20% gap cutoff and every preprocessing statistic come from the training seasons.

3.3 Evidence for the features (training seasons 1950–2019)

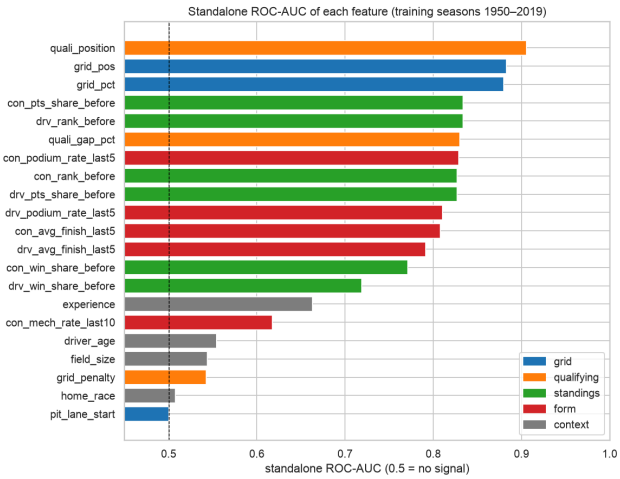


Figure 3.1 Standalone ROC-AUC of each of the 21 candidate features on the training seasons (features where lower is better are flipped; 0.5 = no signal).

- **Starting position is the strongest single signal.** `grid_pos` 0.883 and `grid_pct` 0.880 on all training rows. `quali_position` reaches 0.905, but only on the 37% of rows that have qualifying (1994 on). **On the same rows grid and qualifying are practically identical (0.904 vs 0.905)**: this is why the seasons without qualifying can still be used (D17). Both are kept because they differ exactly in the penalty cases Q1 showed matter.
- **Standings and form come close** (0.79–0.83), and constructor versions are mostly as strong as or stronger than the driver versions: the car matters.
- `quali_gap_pct` 0.830; win shares 0.72–0.77; `experience` 0.663; `con_mech_rate_last10` 0.618; `driver_age` 0.555; `grid_penalty` 0.543 (it works together with qualifying position, Q1).
- **No signal: `pit_lane_start` 0.501, `home_race` 0.508, `field_size` 0.544.** These three are not model inputs (D30). Section 4.6 adds each one back and checks that it never helps significantly.

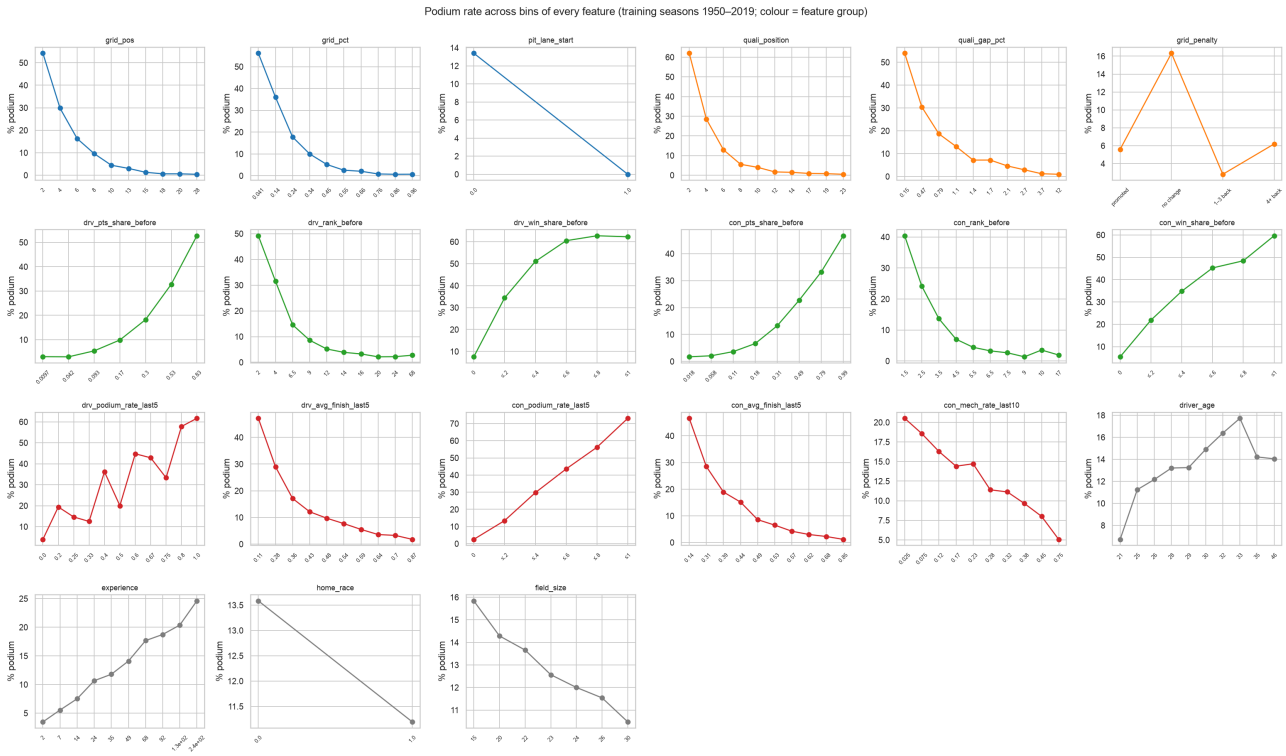


Figure 3.2 Podium rate across bins of every candidate feature (colour = group). Most relationships are monotonic.

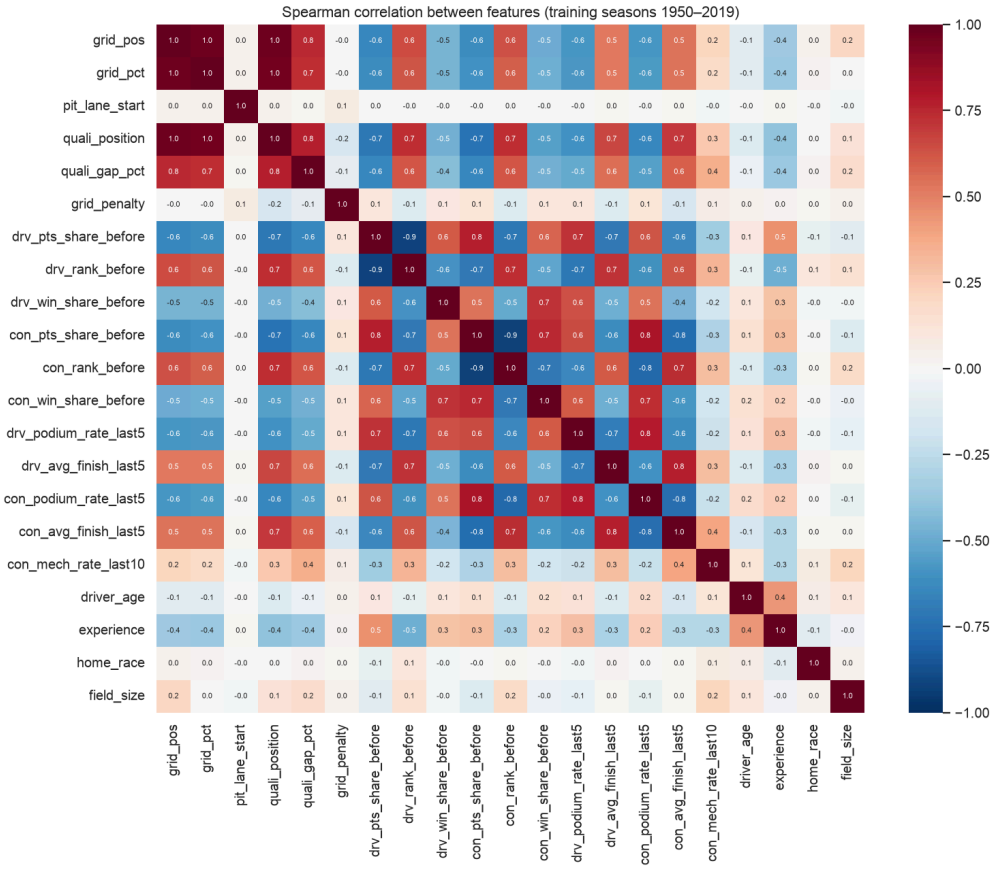


Figure 3.3 Spearman correlation between features. Several features overlap strongly (grid ↔ qualifying position, rank ↔ points share, driver ↔ constructor form), so removing one group costs little, and importance is shared between correlated features.

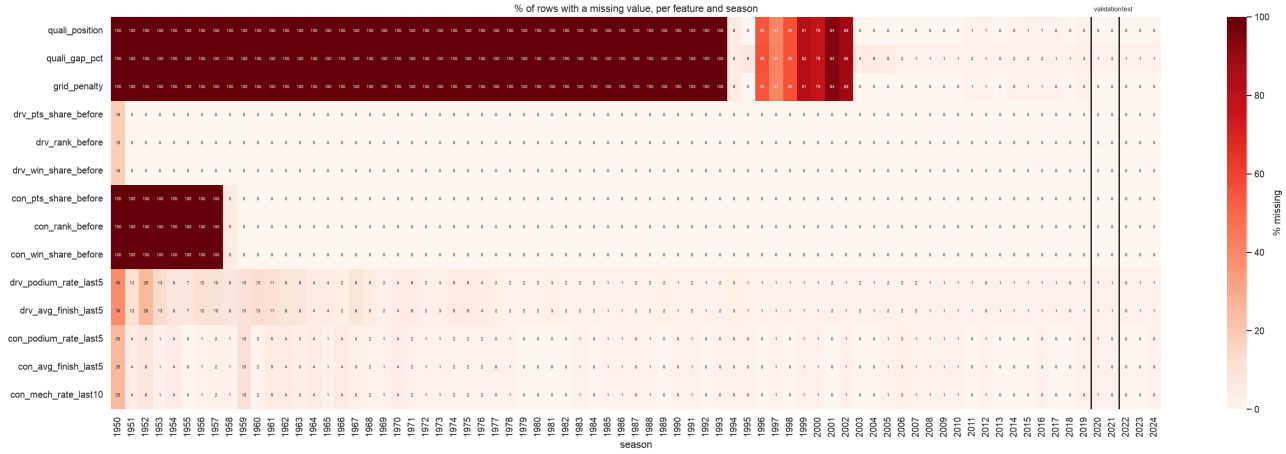


Figure 3.4 % of rows with a missing value, per feature and season. Qualifying is missing for 100% of 1950–1993 rows, 53.6% of 1994–2002, 0.3% of 2003–2019 and none in validation and test. Constructors' standings start in 1958. Form is missing only at a driver's or team's first race.

3.4 Training window and split

Split by season, not by random rows (D18). A random split would put rows of the same race into both training and test, and rows from 2023 into training while testing on 2022. That is information from the future. A season split copies real use: predict the next race from past races. It also tests the 2022 rule change (ground-effect cars).

split	seasons	races	rows	podium rate
train	1950–2019	1,007	22,576	13.4%
validation	2020–2021	39	780	15.0%
test	2022–2024	68	1,359	15.0%

Table 3.2 The split and the class balance per split. An assertion checks that no race is in two splits.

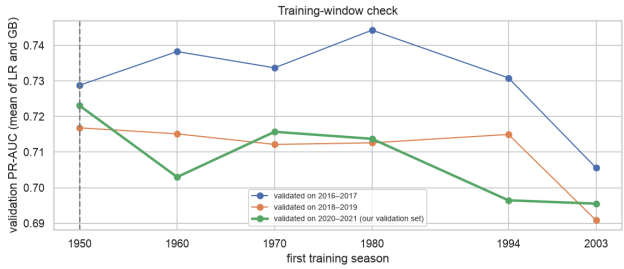


Figure 3.5 Validation PR-AUC by first training season (mean of logistic regression and default gradient boosting), for three validation periods.

Training window 1950–2019 (D17). Leaving seasons out throws data away, so it needs evidence that a shorter window predicts better. We trained from six start years (1950, 1960, 1970, 1980, 1994, 2003), always up to 2019, and scored each on 2020–2021, compared with 1950 by the paired race bootstrap:

- **1950 is best** (mean PR-AUC 0.7230); no later start has a gain interval above 0.
- **Starting in 1994 is significantly worse**: −0.027 (95% CI −0.044 to −0.010); so are 1960 and 2003.
- The logistic regression barely reacts (0.723 from 1950, 0.727 from 1994); gradient boosting loses clearly (0.724 → 0.666): flexible models need the rows.
- On 2016–17 and 2018–19 no start year is best everywhere, and 2003 is lowest in all three periods.

Missing qualifying does not stop the old seasons from being used: grid carries the same signal (Section 3.3), and missing values are filled and flagged (Section 3.5).

3.5 Preprocessing and its effect on the data

- **Imputation.** Missing values are filled with the training median, and a "was missing" indicator is added for the 14 features that have missing values, giving **32 model inputs**. Medians used: qualifying position 11, Q1 gap 1.50%, `grid_penalty` 0, driver podium rate 0, average finish 0.51, team mechanical rate 0.25.
- **Scaling.** The logistic regression and the network also get `StandardScaler`; trees do not need it.
- **Fitted on train only.** Both steps are fitted on the training rows only and then applied unchanged to validation and test (D19).
- **Known caveat.** Missing qualifying is structural: every row before 1994 has no qualifying, while every validation and test row has it. The qualifying indicators therefore also mark the old era (Section 6).

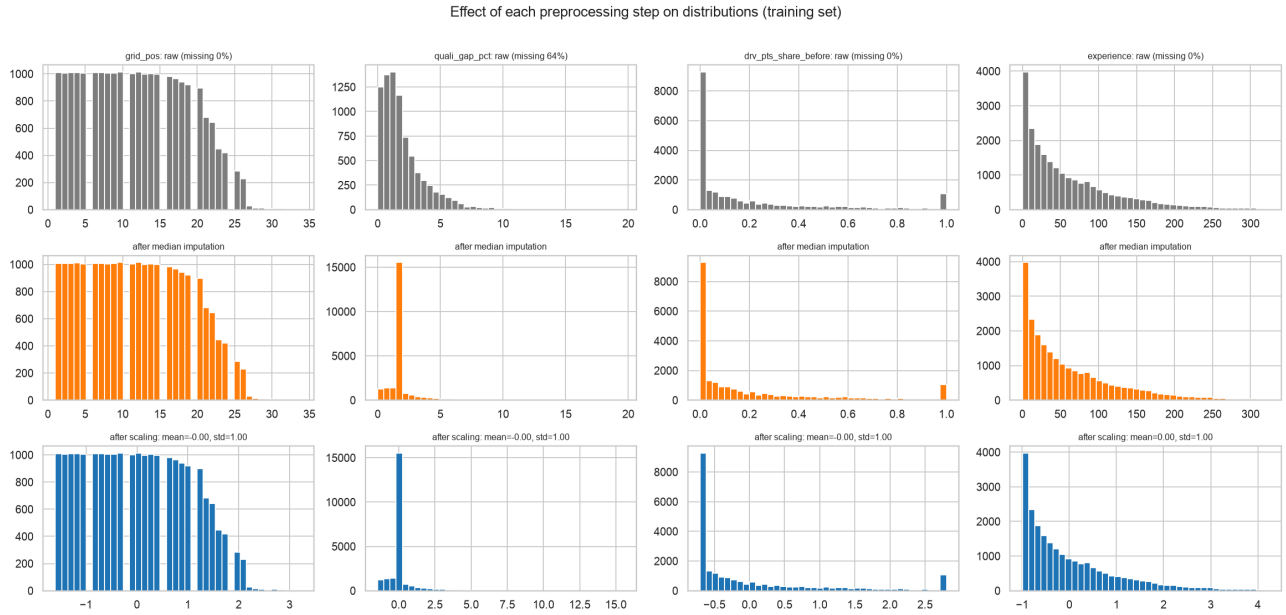


Figure 3.6 Effect of each step on four features (training set): raw, after median imputation, after scaling. Imputation creates a spike at the median for `quali_gap_pct` (64% missing in training); the indicator columns tell the model which values in the spike were really missing. Scaling changes position and spread, not shape.

3.6 Preprocessing-order experiments

Experiments E1–E3 change only the order of two steps, with the same model, rows and threshold rule. They are decided by correctness, not by score: a wrong order can score as high, or higher, precisely because it uses information it should not have.

experiment	order	val PR-AUC	val ROC-AUC	val log-loss	test PR-AUC
E1 · split vs fit first (LR)	split → fit on train (kept)	0.7226	0.9324	0.2245	0.7219
	fit on all data → split (leaky)	0.7230	0.9326	0.2245	0.7219
E2 · impute vs scale first	LR: impute → scale (kept)	0.7226	0.9324	0.2245	0.7219
	LR: scale → impute	0.7214	0.9319	0.2250	0.7223
	NN: impute → scale (kept)	0.7158	0.9290	0.2274	0.7112
	NN: scale → impute	0.7227	0.9279	0.2289	0.7034
E3 · derive vs impute first (LR)	derive → impute (kept)	0.7226	0.9324	0.2245	0.7219
	impute → derive	0.7208	0.9296	0.2268	0.7192

Table 3.3 Preprocessing-order experiments (validation chooses; test for reference only).

- **E1, data leakage through preprocessing.** Fitting the imputer and scaler on all seasons barely changes the scores (0.7226 vs 0.7230). But the statistics it learns come from the future: the `experience` median moves 41 → 45 and the Q1-gap median 1.50 → 1.34. The correct order costs nothing, so it is kept.
- **E2, order of imputation and scaling.** The logistic regression is unaffected (0.7226 vs 0.7214). The network scores 0.007 higher on validation with scale → impute (one run per order, about the size of the network's seed spread, Section 4.4), but its validation log-loss and ROC-AUC are worse and its test PR-AUC lower. Impute → scale is kept: it is the standard order, and the model sees exactly mean 0 and standard deviation 1 (Figure 3.7).

- **E3, derived features.** Imputing `quali_position` before computing `grid_penalty` invents a penalty for all 14,228 training rows without qualifying data, and is worse on validation PR-AUC, ROC-AUC and log-loss. A feature value must be real or marked as missing, so derived features are computed first and imputed afterwards.

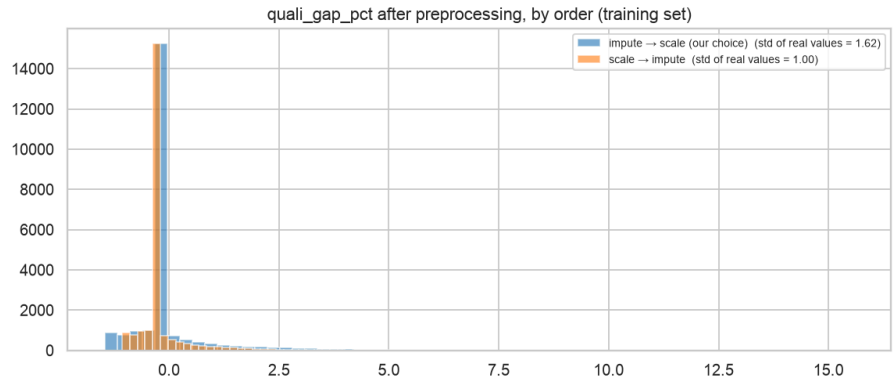


Figure 3.7 `quali_gap_pct` after preprocessing in the two E2 orders (training set).

- **E4, how to fill missing qualifying.** Since training starts in 1950, qualifying is missing for 63% of training rows. Three options were compared on validation, against our choice by the paired race bootstrap:

missing-qualifying option	LR	GB default
median + missing indicator (kept)	**0.7226**	**0.7235**
median, no indicator	0.7103 (–0.023 to –0.003)*	0.7017 (–0.048 to +0.010)
grid fallback (quali position = grid slot) + indicator	0.7154 (–0.014 to –0.001)*	0.6633 (–0.087 to –0.013)*

Table 3.4 Experiment E4, validation PR-AUC (95% interval of the change against our choice; * = excludes 0).

The median value itself is only a placeholder: the indicator tells the model the value was invented, so the model treats those rows as a group that relies on the grid. Without the indicator the logistic regression loses significantly. Filling qualifying with the grid slot is worse for both models (gradient boosting –0.06): it makes qualifying identical to the grid and erases the penalty signal Q1 found. Median + indicator is kept.

4. Predictive modeling

Every model predicts, for one driver on the grid of one race, the probability of finishing in the top three (`positionOrder ≤ 3`). It uses only information known after qualifying and before the start.

4.1 Set-up: same rows, same features, choices on validation only

All models are trained and evaluated on the same modeling table: one row per driver per race, and the same 18 pre-race features in five groups (Section 3). Rows are split by season:

split	seasons	races	rows	podium rate	used for
train	1950–2019	1,007	22,576	13.4%	fitting the models and the preprocessing
validation	2020–2021	39	780	15.0%	thresholds, hyperparameters, early stopping, model comparison
test	2022–2024	68	1,359	15.0%	scored once per model; never used to choose anything

Table 4.1 Splits. The base rate (≈ 0.15) is also the PR-AUC of a random model, so every PR-AUC below should be read against it.

Preprocessing (Section 3.5) is fitted on the training rows only: median imputation with 14 "was missing" indicators (32 inputs), plus `StandardScaler` for the logistic regression and the network.

4.2 Metrics, threshold and uncertainty

- **Primary metric: PR-AUC** (average precision). Podiums are rare (≈ 15%), and PR-AUC scores how clean the top of the ranking is. A random model's PR-AUC is the base rate (0.15), while its ROC-AUC is 0.5.
- **Secondary metric: ROC-AUC**, for the overall separation of podium from non-podium.
- **F1, precision and recall** at one decision threshold per model: the threshold that maximises F1 on validation, then used unchanged on train and test (D22).
- **Accuracy is not used:** a model that always says "no podium" has accuracy 0.850 on validation and test and recall 0.
- **Brier score** checks calibration: whether a predicted 0.3 really means about 30%. This matters because the inference functions return a probability.
- **Uncertainty.** Validation has only 39 races, so each important number gets a 95% interval from 1,000 bootstrap resamples of races (drivers in one race are not independent). A difference between two models is resampled paired, with both scored on the same redrawn races (D23).

4.3 The models and how they work

model	how it works	settings, and how they were chosen
0. Grid rule reference, no learning	Score = 1 / grid slot. It shows how much the learned models add on top of starting position.	Validation threshold 0.333, exactly "podium if grid ≤ 3".
1. Logistic regression baseline	A weighted sum of the scaled inputs turned into a probability by the sigmoid. Each weight is the change in log-odds per standard deviation of that input, with the others held fixed.	scikit-learn default C = 1, a mild L2 penalty; the baseline needs no tuning.

2. Random forest	400 trees, each on a bootstrap sample of rows, each split choosing from $\sqrt{32} \approx 6$ inputs. The probability is the average podium share of the leaves the row falls into.	Leaf size {5, 10, 20, 50, 100} chosen on validation → 100. Class weights were compared in the same grid but not used (D20).
3. Gradient boosting (default)	Small trees added one after another; each fits the errors of the current model, with a learning rate. Inputs are binned into histograms.	scikit-learn defaults: learning rate 0.1, 100 trees, 31 leaves, minimum leaf 20.
4. Gradient boosting (tuned)	The same algorithm.	A 54-point grid (learning rate × trees × leaves × minimum leaf) on validation. Chosen: learning rate 0.01, 50 trees, 7 leaves, minimum leaf 20.
5. Shallow neural network	32 inputs → 32 ReLU → 16 ReLU → 1 sigmoid (1,601 weights), Keras; Adam, binary cross-entropy, dropout 20% and L2 1e-4.	Early stopping on validation PR-AUC (patience 15, best epoch kept); 3 seeds (42, 1, 2).

Table 4.2 The six modeling attempts, all on the same rows and features.

Early stopping is off for gradient boosting (D28). scikit-learn's default switches it on above 10,000 rows and holds out a random 10% of rows, which would mix drivers of the same race and make results depend on the seed. The number of trees is tuned on validation instead.

Class imbalance (D20). No model uses class weights. Weights make the rare class count more, but they push every predicted probability up, and the inference functions return a probability (van den Goorbergh et al., 2022). The forest grid shows weights do not help either: the best weighted forest scores 0.7310 on validation against 0.7334 unweighted.

4.4 Results: training, validation and test

model	PR-AUC	95% CI	ROC-AUC	F1	precision	recall	Brier	threshold
0. grid rule	0.639	0.541–0.745	0.910	0.692	0.692	0.692	0.083	0.333
1. logistic regression	**0.723**	**0.612–0.836**	**0.932**	**0.699**	**0.613**	**0.812**	**0.067**	**0.241**
2. random forest	0.733	0.624–0.840	0.932	0.708	0.643	0.786	0.067	0.279
3. gradient boosting (default)	0.724	0.627–0.820	0.924	0.693	0.702	0.684	0.070	0.398
4. gradient boosting (tuned)	0.736	0.634–0.834	0.926	0.704	0.669	0.744	0.094	0.212
5. shallow NN (32–16)	0.716	0.607–0.827	0.929	0.703	0.641	0.778	0.068	0.265

Table 4.3 Validation (2020–2021), 39 races, random-model PR-AUC 0.150. Shaded row: the recommended model (Section 4.5).

model	PR-AUC	95% CI	ROC-AUC	F1	precision	recall	Brier	podiums / race
0. grid rule	0.645	0.590–0.708	0.913	0.647	0.647	0.647	0.081	3.00
1. logistic regression	**0.722**	**0.658–0.785**	**0.937**	**0.673**	**0.546**	**0.877**	**0.068**	**4.82**
2. random forest	0.722	0.664–0.779	0.940	0.672	0.563	0.833	0.069	4.44
3. gradient boosting (default)	0.693	0.634–0.754	0.930	0.646	0.667	0.627	0.071	2.82
4. gradient boosting (tuned)	0.696	0.635–0.759	0.936	0.680	0.622	0.750	0.096	3.62

5. shallow NN (32–16)	0.711	0.651–0.769	0.937	0.667	0.569	0.804	0.070	4.24

Table 4.4 Unseen test seasons (2022–2024), 68 races. Each model uses its validation threshold.

model (training seasons 1950–2019)	PR-AUC	ROC-AUC	F1	Brier
0. grid rule	0.496	0.883	0.543	0.091
1. logistic regression	0.570	0.900	0.582	0.078
2. random forest (out-of-bag)	0.573	0.901	0.585	0.078
3. gradient boosting (default)	0.765	0.959	0.686	0.058
4. gradient boosting (tuned)	0.574	0.899	0.588	0.092
5. shallow NN (32–16)	0.588	0.906	0.588	0.077

Table 4.5 Training seasons. The random forest's training score is its out-of-bag estimate; its in-sample score (0.599) is inflated by memorised rows.

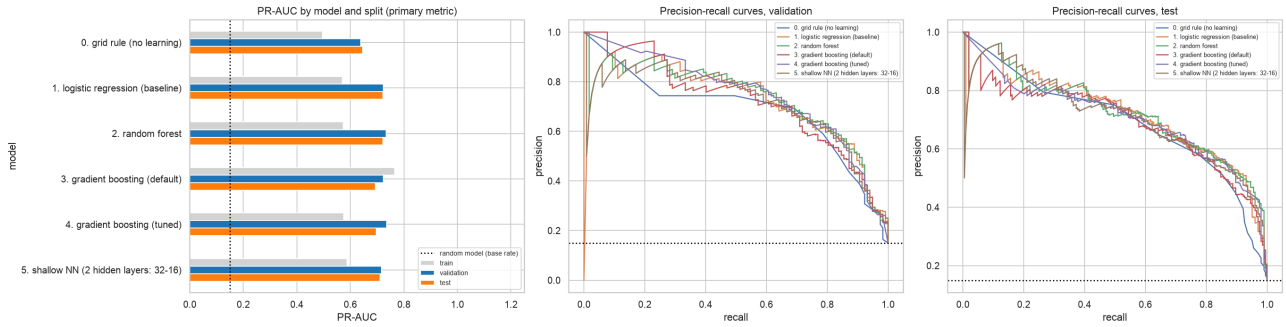


Figure 4.1 Left: PR-AUC per model on train, validation and test (dotted line = random model). Middle and right: precision-recall curves on validation and test.

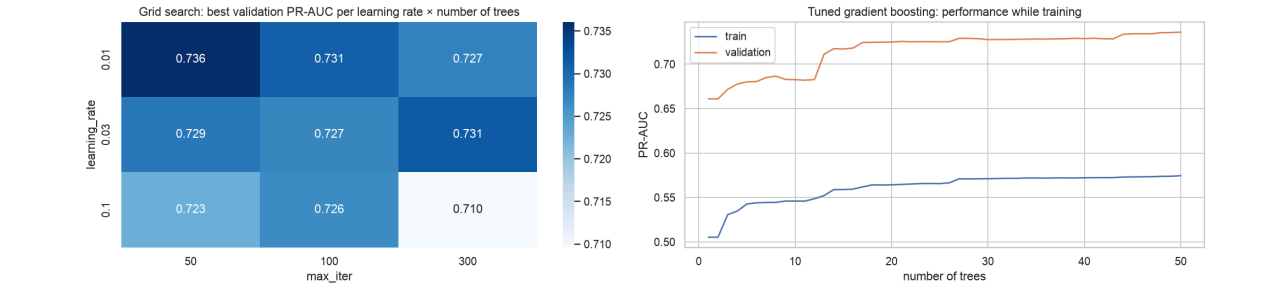


Figure 4.2 Gradient boosting. Left: best validation PR-AUC per learning rate × number of trees in the 54-point search. Right: the tuned model's train and validation PR-AUC as trees are added.

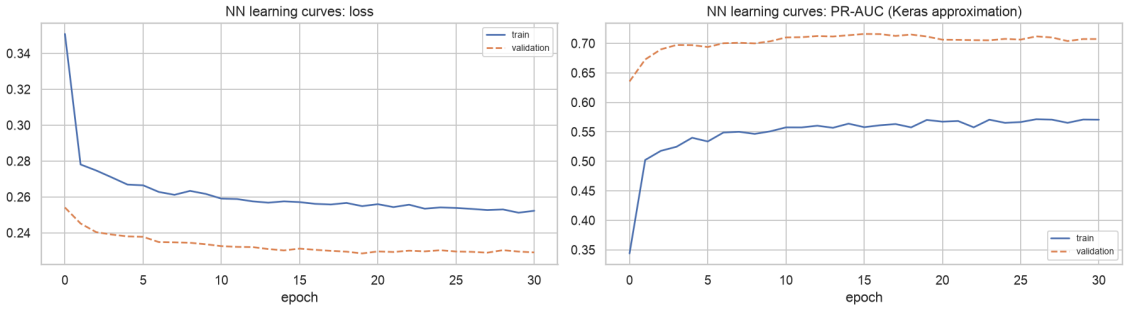


Figure 4.3 Shallow network, seed 42: loss and PR-AUC per epoch. Training stopped 15 epochs after the best validation epoch, and the best epoch's weights were kept.

What training and validation show

- **Overfitting is visible, then removed.** Default gradient boosting fits the training seasons far better than anything else (train PR-AUC 0.765) but reaches 0.724 on validation. The tuned settings learn slowly with very simple trees (0.574 train, 0.736 validation). The best point is the corner of the grid (slowest learning, fewest trees, smallest trees), which suggests an even simpler model would do about as well.
- **Validation is above training for every model**, even for the grid rule (0.496 train vs 0.639 validation), which learns nothing, so this is not leakage. The old seasons are much harder to rank: grid-rule PR-AUC 0.39–0.46 per decade before 1994, against 0.68–0.70 in 2014–2019 (bigger fields, frequent breakdowns). The baseline beats the grid rule in every period.
- **Seed spread.** Over 3 seeds, the forest's validation PR-AUC ranges 0.723–0.733 (mean 0.728; test mean 0.723), the network's 0.716–0.721 (mean 0.719; test mean 0.697). Differences between models smaller than this mean little.

4.5 Model comparison: no model beats the baseline, so the logistic regression is recommended

> **Rule (validation only, D24).** Rank the models by validation PR-AUC (mean over seeds for the forest and the network). A more complex model is better than the logistic-regression baseline only if the paired bootstrap interval of (model − baseline) lies entirely above 0. All five models are kept, explained (Section 5) and served (Section 6); the comparison decides which one we recommend.

model	validation PR-AUC	95% CI of difference vs baseline	better than baseline
4. gradient boosting (tuned)	0.7361	−0.023 to +0.040	no
2. random forest (3-seed mean)	0.7276	−0.010 to +0.024	no
3. gradient boosting (default)	0.7235	−0.040 to +0.033	no
1. logistic regression (baseline)	**0.7226**	**—**	**—**
5. shallow NN (3-seed mean)	0.7190	−0.043 to +0.024	no

Table 4.6 The comparison. Baseline − grid rule: 95% interval 0.031 to 0.144, entirely above 0.

- **No model earns its extra complexity.** Every interval includes 0. This test even favours tuned gradient boosting, the best of 54 tries on the same 39 races, and it still does not clear the bar.
- **The baseline clearly beats the grid rule** (0.031 to 0.144): standings, form and qualifying pace add real information on top of starting position.
- **The test seasons confirm the recommendation was not costly.** Logistic regression 0.722, random forest 0.722, network 0.711, tuned and default gradient boosting 0.696 and 0.693, all well inside each other's intervals (about ±0.06).
- **Why the logistic regression is the recommended model** (used first in Sections 5 and 6): it is as good as any other model, the simplest, deterministic, and among the best calibrated (validation Brier 0.067, with the random forest 0.067 and the network 0.068; tuned gradient boosting 0.094 keeps its predictions near the average).

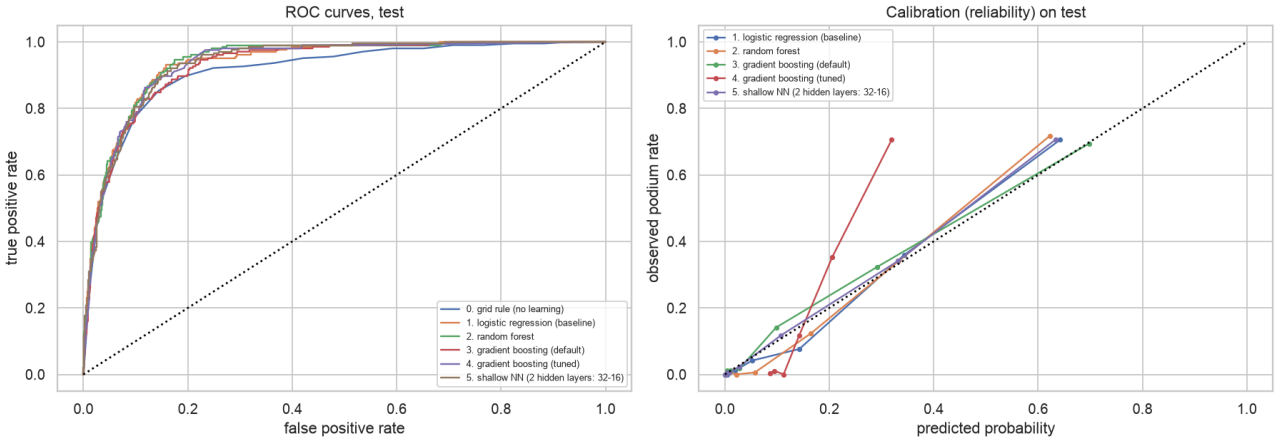


Figure 4.4 Test seasons. ROC curves and calibration per model; points on the diagonal are well calibrated (drawn on test, for reference only).

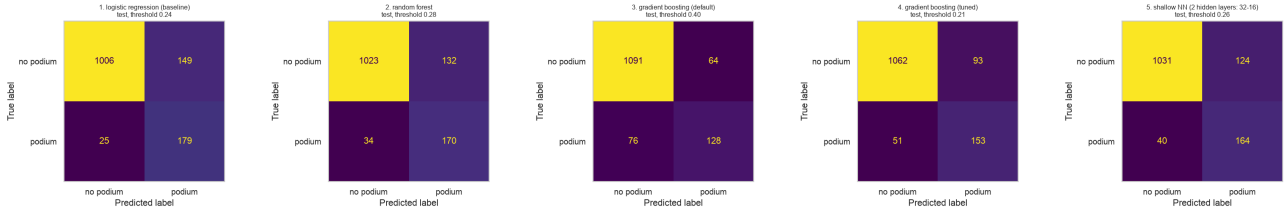


Figure 4.5 Test confusion matrices of all five models at their validation thresholds.

What the comparison shows

- **The learned models clearly beat the grid rule, but not each other.**
- **Calibration.** Without class weights, the logistic regression, the random forest, the network and default gradient boosting all lie close to the diagonal. Tuned gradient boosting is under-confident at the top: its slow learning keeps predictions near the base rate.
- **Race top-3 hit rate.** Taking the three highest scores in each race, the learned models catch about as many podium finishers as the first three on the grid (validation 0.675 vs 0.692; test 0.637–0.662 vs 0.647). Their advantage is not in reordering the front of one grid, but in knowing *how likely* each front-runner is to podium (a front-row start in the dominant car vs a midfield car), which PR-AUC and the Brier score reward.
- **Decisions at the threshold.** The logistic regression's F1-optimal threshold is low (0.241), so it predicts about 4.0 podiums per race on validation and 4.8 on test, more than the 3 possible. This follows from scoring each driver on their own (Section 4.7).

4.6 Feature-group ablation

Section 3 justified each feature before training. The ablation measures each group's contribution after training: we remove one group at a time and retrain with rows, hyperparameters and preprocessing unchanged, for **all five models** (the forest and the network averaged over 3 seeds), each change with a paired race-bootstrap interval. **Rule (D30):** a group or feature is removed from a model only if removing it improves that model's validation PR-AUC with an interval entirely above 0.

variant (validation)	LR	RF	GB default	GB tuned	NN
all 18 features (PR-AUC)	0.723	0.730	0.724	0.736	0.723
without grid	−0.003	**−0.039***	+0.007	−0.035	+0.004
without qualifying	−0.015	−0.004	**−0.045***	−0.014	+0.004

without standings	−0.009	−0.016	**−0.054***	**−0.029***	−0.004
without form	+0.015	+0.004	+0.008	−0.003	+0.006
without context	**−0.009***	−0.004	−0.030	0.000	−0.003
grid group only	**−0.084***	**−0.091***	**−0.085***	**−0.100***	**−0.084***

Table 4.7 Change in validation PR-AUC against all 18 features (negative = the removed group helped). * = 95% paired race-bootstrap interval excludes 0. Test-season changes are in notebook §10.

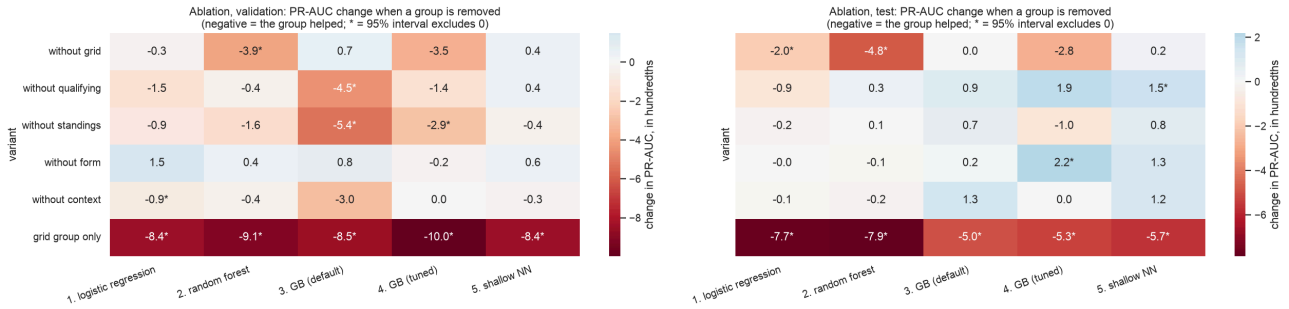


Figure 4.6 The same changes on validation and test, in hundredths of PR-AUC, for all five models.

- **Starting position is the backbone, but not enough on its own.** Keeping only the two grid features loses 0.08–0.10 PR-AUC on validation for every model (0.05–0.08 on test), all significant. Together, the other groups add real information on top of the grid.
- **Removing one group mostly changes little, because the groups overlap** (grid ↔ qualifying, standings ↔ form). The significant losses are model-specific: grid for the random forest, qualifying and standings for default gradient boosting, standings for tuned gradient boosting, context (through `experience`) for the logistic regression.
- **Form shows no measurable contribution as a group** (its removal even raises the logistic regression by +0.015, interval −0.011 to +0.033): the standings carry the same season-long strength.
- **Single features.** Removing `experience` alone hurts the logistic regression significantly (−0.009, interval −0.018 to −0.002); removing `grid_penalty` or `driver_age` is within noise for every model.
- **Adding back the three no-signal candidates** (`pit_lane_start`, `home_race`, `field_size`, one at a time and all three together) never improves any model significantly; for the logistic regression every change is within ±0.001. This confirms D30.
- **No group or feature meets the removal rule in any model**, so all five models keep the 18 features.

4.7 Strengths and limitations

model	strengths	limitations
logistic regression	simple, fast, deterministic; readable weights; well-calibrated probabilities	each feature moves the log-odds along a straight line; interactions (e.g. "pole matters more at Monaco") must be built by hand
random forest	non-linear effects and interactions; robust to outliers and unscaled inputs	large; in-sample score above out-of-bag (0.599 vs 0.573); moves with the seed
gradient boosting	usually strong on tabular data; handles interactions	many hyperparameters; overfits with fast learning and complex trees (default: 0.765 train vs 0.724 validation); tuning on 39 races is noisy; under-confident once tuned

shallow network	can approximate smooth non-linear functions	needs scaling and regularisation; moves with the seed; harder to explain; no advantage on ~22,000 tabular rows

Table 4.8 Strengths and limitations per model.

Limitations shared by all models.

- **Each driver is scored independently.** Nothing forces exactly three podiums per race, so the yes/no output predicts 3–5 per race. A race-level model such as Plackett–Luce or a conditional logit (Henderson & Kirrane, 2018) would solve this; it is out of scope here.
- **No race-day information.** There is no weather, tyre strategy, safety car, crash or breakdown data, so upsets cannot be foreseen (Section 5.4).
- **Small evaluation sets.** Validation has 39 races, so model and threshold choices carry noise, and validation scores are slightly optimistic because validation was used for several choices. Test has 68 races, and its intervals are about ±0.06 wide.
- **Era shift.** The test seasons started a new set of technical rules, and the models still transfer to them. A future rule change may not behave the same way.

5. Explainability (XAI)

> **Explanations are not causal effects.** A feature being important means the model relies on it to predict. It does not mean the feature causes podiums. For example, `con_pts_share_before` is important because strong teams both score points and finish on the podium; giving a team points would not make its cars faster. Correlated features also share importance. Every statement in this section describes the models' behaviour, not the sport.

5.1 What we explain, and on which data

We explain **all five models**, leading with the recommended one, the logistic regression (Section 4.5); the other four serve as a cross-check of whether very different models rely on the same information. All explanations are computed on the unseen test seasons (2022–2024); explaining does not choose anything, so this does not leak.

- **Global:** permutation importance per feature and per feature group; SHAP summary (beeswarm) and mean |SHAP| per model.
- **Local:** a SHAP waterfall and LIME for the same single prediction, plus each model's most confident wrong prediction.
- **SHAP set-up.** One model-agnostic `PermutationExplainer` for all five models, in probability units, on the original feature values. The background ("a typical driver") is 100 training rows from **2003–2019**, the seasons with complete qualifying data, like every row we explain (D26). For every explained row the base value plus the contributions equals the model's probability (largest gap 1e-8).

5.2 Permutation importance (global)

We shuffle one feature's values across the test rows, which breaks its link to the outcome, and measure how much test PR-AUC drops (20 repeats). We also shuffle whole groups together, which is fairer for correlated features: shuffling only `grid_pos` still leaves `grid_pct` intact.

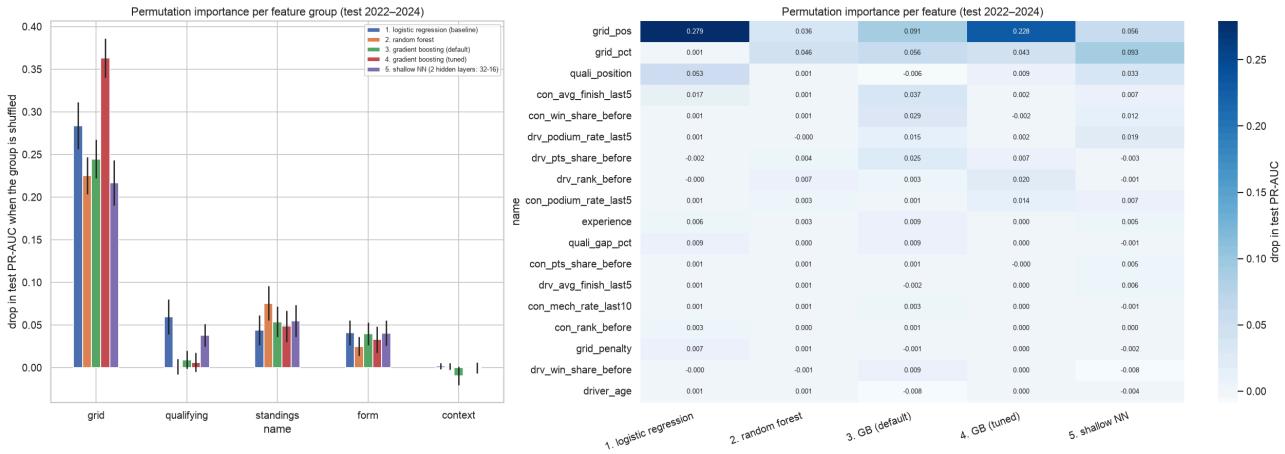


Figure 5.1 Drop in test PR-AUC when a feature or a whole group is shuffled, for each model (mean of 20 shuffles).

group shuffled	LR	RF	GB default	GB tuned	NN
grid	0.284	0.225	0.245	0.363	0.217
qualifying	0.060	0.001	0.009	0.006	0.038

standings	0.044	0.076	0.054	0.049	0.055
form	0.041	0.025	0.040	0.033	0.041
context	0.002	0.002	−0.009	0.000	−0.000

Table 5.1 Drop in test PR-AUC when a whole group is shuffled.

- **Every model depends overwhelmingly on where the car starts** (a drop of 0.22–0.36), then on season-long strength and recent form.
- **The logistic regression also uses qualifying** (0.060); the tree models barely do, because for them the grid already carries the same ranking.
- **Context adds nothing measurable** in any model, in line with the ablation.
- **Single features:** `grid_pos` is clearly above 0 in every model; most single standings and form features are not, because a correlated partner takes over when one is shuffled. This is why the group view is the fairer one.

5.3 SHAP (global)

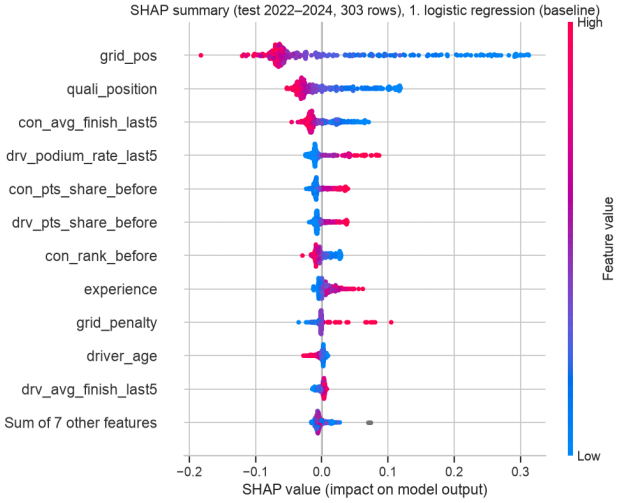


Figure 5.2 SHAP summary of the logistic regression on 303 test rows. Each dot is one driver-race; horizontal position is how much that feature moved the probability; colour is the feature's value (red = high).

input (scaled)	weight
grid_pos	−1.482
missing: quali_gap_pct	+0.880
quali_position	−0.333
missing: grid_penalty	−0.282
missing: quali_position	−0.282
con_avg_finish_last5	−0.205
con_rank_before	−0.163
drv_podium_rate_last5	+0.161
con_pts_share_before	+0.130
drv_pts_share_before	+0.107

Table 5.2 The largest logistic-regression weights: change in log-odds per standard deviation, the others held fixed.

share of mean |SHAP|	LR	RF	GB default	GB tuned	NN
grid	37.6%	38.7%	35.9%	58.8%	34.3%
standings	16.5%	33.0%	25.2%	25.0%	21.8%
form	20.0%	19.3%	21.0%	14.1%	27.6%
qualifying	20.2%	8.1%	6.7%	2.1%	11.6%
context	5.7%	0.9%	11.2%	0.0%	4.8%

Table 5.3 Each model's total mean |SHAP| on the explained test rows, split by feature group.

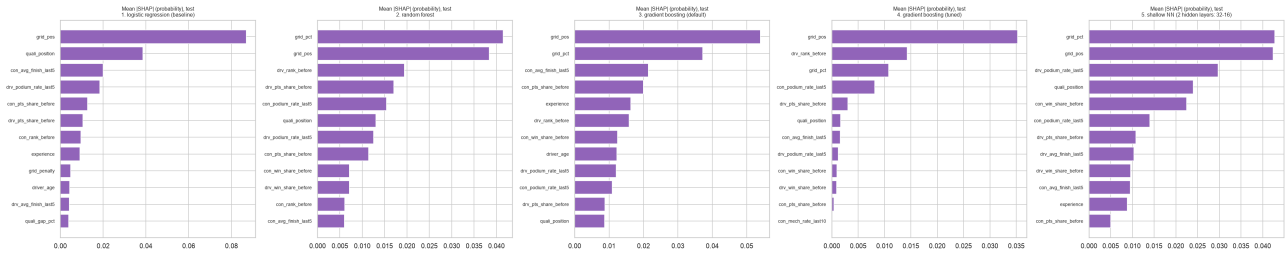


Figure 5.3 Mean |SHAP| (probability) per feature on test, for every model.

- **Starting position dominates in all five models** (34–59% of the total); `grid_pos` or `grid_pct` is the top feature everywhere. Five very different models rely on the same information: where the car starts, and how strong the driver and team have been.
- **Directions are sensible.** A slot near the front, a strong constructor points share and a recent podium habit push the prediction up; a poor team average finish pushes it down.
- **The missing-qualifying indicators have large weights of opposite sign** (+0.88, −0.28, −0.28). They cancel when all three are missing together, the usual pattern before 1994. Because the SHAP background now contains only rows with qualifying data, like the test rows, these indicators no longer show up as a large artificial contribution (D26).
- Single weights of correlated inputs (e.g. `grid_pos` −1.48 vs `grid_pct` −0.01) split shared information; read the group totals, not single numbers.

5.4 Local SHAP: one prediction explained

We explain a well-known test case: **Max Verstappen at the 2023 Belgian GP**. He was fastest in qualifying, took a 5-place gearbox penalty, started 6th, and won. Inputs: grid 6 of 20, qualifying position 1, `grid_penalty` 5; leading both championships (points share 1.00, win share 0.82, team win share 1.00); podiums in all of his last 5 races; no team mechanical failures in the last 10 races.

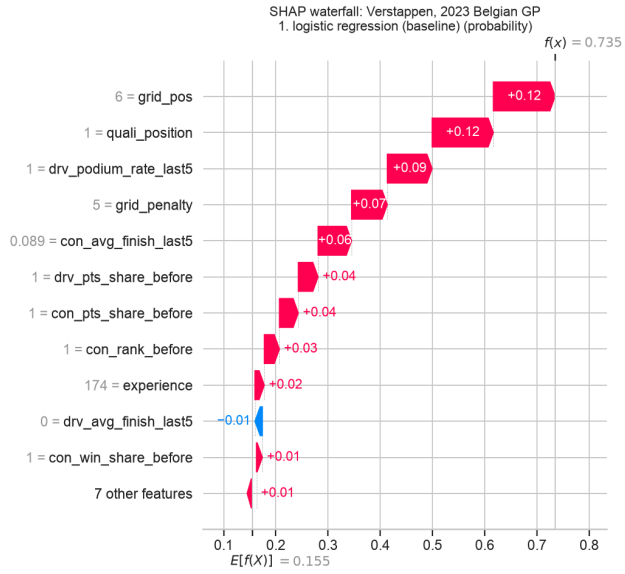


Figure 5.4 SHAP waterfall of the logistic regression, Verstappen at the 2023 Belgian GP (probability).

model	probability	decision
logistic regression	**0.735**	**podium**
random forest	0.490	podium
gradient boosting (default)	0.538	podium
gradient boosting (tuned)	0.230	podium
shallow NN	0.513	podium

Table 5.4 All five models say "podium" at their own thresholds. He won.

LR, grid slot given	probability
P1 (penalty 0)	0.840
P6 (actual, penalty 5)	0.735
P10 (penalty 9)	0.623

Table 5.5 Model what-if (model behaviour, not a causal claim).

- **The logistic regression:** base value 0.155 (the average prediction over the background drivers) + contributions 0.580 = **0.735**. Every group pushes up: qualifying +0.176, form +0.137, standings +0.125, grid +0.119, context +0.022. The largest single contributions are `grid_pos` (+0.118),

`quali_position` (+0.117) and `drv_podium_rate_last5` (+0.086): his pole lap is credited, even though he started 6th.

- **The models agree on the decision but not on the number** (0.230–0.735), because they are calibrated differently: tuned gradient boosting keeps all predictions near the average. The tree models credit his standings most (forest: standings +0.184).

The models' most confident mistakes

For the logistic regression, the forest and the network, the highest probability given to a driver who did **not** podium is the same row: **Verstappen at the 2024 Australian GP**, on pole and leading both championships (LR 0.849). He retired with a brake failure. For the LR every group pushed up (grid +0.313, form +0.139, standings +0.118, qualifying +0.101). Nothing in the pre-race data could have announced a component failure: the models rank the field well but cannot foresee breakdowns or crashes (Section 4.7).

5.5 LIME for the same prediction

LIME creates perturbed copies of the row, asks the model about each, and fits a small weighted linear model around that point. It describes features by ranges (e.g. `grid_pos ≤ 6`), changes features independently (so some synthetic rows are impossible), and is random, so we ran it with 5 seeds.

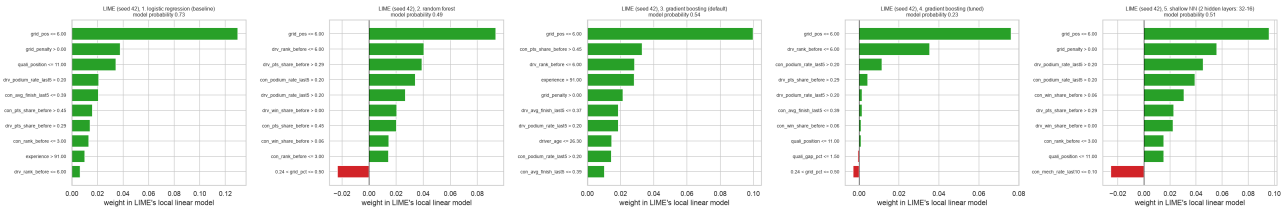


Figure 5.5 LIME for the same prediction, every model (seed 42).

LIME condition (LR)	mean weight	seeds in top 10
grid_pos ≤ 6	+0.131	5/5
grid_penalty > 0	+0.041	5/5
quali_position ≤ 11	+0.032	5/5
drv_podium_rate_last5 > 0.20	+0.022	5/5
con_avg_finish_last5 ≤ 0.39	+0.022	5/5

Table 5.6 LIME weights for the logistic regression, averaged over 5 seeds. Local fit R^2 0.69–0.70; LIME's own prediction 0.28 against the model's 0.735.

- **Direction agrees with SHAP.** For every model `grid_pos ≤ 6` is the largest weight in all 5 seeds, followed by a qualifying or standings condition with the same sign SHAP gave.
- **But LIME fits this point only approximately.** Its R^2 is 0.70 for the logistic regression and 0.34–0.52 for the forest, default gradient boosting and the network, and its own prediction (0.28 for the LR) is far from the model's. LIME draws synthetic rows from the whole training distribution, mostly midfield cars with near-zero podium chances, and one straight line through them cannot follow the steep rise near this row. (Tuned gradient boosting fits best, R^2 0.88, only because its output barely moves.)
- **SHAP is therefore the main local explanation**: it adds up exactly to each model's output.

5.6 Summary of the explanations

- All methods and all five models agree that **starting position** is what the models rely on most, followed by **season-long strength** (standings) and **recent form**; the logistic regression also uses qualifying pace.
- Group-level views are the trustworthy ones: correlated features share and trade importance.
- The local explanations show what the models get right (a penalised pole-sitter in the dominant car is still a likely podium) and what they cannot know (a brake failure from pole).
- These explain the models' reasoning, not what causes a podium.

6. Inference functions

Each model has its own function: `predict_podium_lr`, `predict_podium_rf`, `predict_podium_gb_default`, `predict_podium_gb_tuned` and `predict_podium_nn` (D27). Each takes one driver's pre-race information as a dictionary and returns the podium probability, a yes/no decision at that model's validation-tuned threshold, and the list of missing features. **`predict_podium_lr` is the recommended one** (Section 4.5). All five share one input check and use their model's own fitted preprocessing, so missing values are treated exactly as in training (training median plus the "was missing" indicator).

- **Required:** `grid_pos` and `field_size`. A raw grid of 0 is read as a pit-lane start and moved to the last slot (D7).
- **Derived when absent:** `grid_pct` from slot and field size, and `grid_penalty` from slot and qualifying position.
- **Rejected with an error:** unknown feature names (a typo would otherwise silently count as "missing"), a grid slot outside the field or not a whole number, shares and rates outside 0–1, and a `grid_pct` that contradicts the slot.

demo record	LR (0.241)	RF (0.279)	GB default (0.398)	GB tuned (0.212)	NN (0.265)
1. complete: Verstappen, 2023 Belgian GP (he won)	0.735 podium	0.490 podium	0.538 podium	0.230 podium	0.513 podium
2. partially missing: Guanyu Zhou, 2022 Bahrain GP, his first race (own form missing)	0.004 no	0.002 no	0.001 no	0.087 no	0.001 no
3. the same debutant from the pit lane (raw grid 0)	0.002 no	0.001 no	0.000 no	0.087 no	0.001 no

Table 6.1 The two required demonstrations (a complete and a partially-missing record) plus a pit-lane variant, from every function (threshold in brackets). The partial record is real: a debutant's grid, qualifying and team data are known, but he has no past races, so his own form is missing, exactly as for every debutant in training. He did not podium.

removed from Verstappen's complete record	LR	NN
nothing	0.735	0.513
qualifying group	0.603	0.676
form group	0.539	0.851
qualifying and form	0.391	0.833

Table 6.2 What happens when blocks of a veteran's record are missing (all five models in notebook §12).

- **For the recommended logistic regression, removing information lowers the probability step by step** (0.735 → 0.603 → 0.539 → 0.391): without qualifying it no longer knows he was fastest, and without form it no longer sees his podium run. That is the behaviour a user would expect.
- **The network moves the other way** (0.513 → 0.833). In training, a row without qualifying is almost always a pre-1994 row, and a row without form is a debutant; a veteran with both missing is a pattern the network never saw, so its output is not reliable there.
- **The lesson:** an imputed prediction is only as good as the training data's examples of that missing pattern. That is why every function returns the list of missing features, and why the realistic partial

demo (a debutant) is the one a user will actually meet.

7. Known limitations

- **Historical qualifying-coverage drift.** Qualifying data starts in 1994 and is patchy until 2002 (100% of 1950–1993 rows and 53.6% of 1994–2002 rows have no qualifying position). The models learn qualifying mostly from 2003 on, and the missing indicators double as an "old era" marker, which makes records with qualifying missing less reliable (Section 6). Keeping these seasons was tested, not assumed: no later start year predicts better, and starting in 1994 is significantly worse (Section 3.4); filling qualifying with the grid slot instead of the median was also worse (notebook §8.5).
- **Era sensitivity.** Rules, reliability and field sizes changed a lot. A pole start converts 51%, 65% or 81% depending on the era (Figure 1.6), and the old seasons are much harder to rank. The test seasons start a new set of technical rules and the models still transfer, but a future rule change may not behave the same way.
- **Small evaluation sets.** Validation has 39 races and test 68, so PR-AUC intervals are about ±0.06–0.11 wide. Differences between our learned models are inside them. Validation scores are slightly optimistic, because validation was used for several choices.
- **Independent predictions.** Each driver is scored alone. Nothing forces exactly three podiums per race, so the yes/no output of the logistic regression predicts about 4–5 per race. A race-level model (Plackett–Luce or a conditional logit) would fix this; it is out of scope for this milestone.
- **No race-day information.** Weather, tyre strategy, safety cars, crashes and breakdowns are not in the data and cause many upsets. The most confident miss is a pole-sitter's brake failure (Section 5.4).
- **Team identity in the features.** A renamed team gets a new `constructorId`, so its constructor form history restarts at the rename. Q3 groups teams by lineage; the features do not.
- **Data quirks documented but not fully fixable:** shared drives in the 1950s (the starting car cannot always be identified, D4, and one race still has four podium rows); a few vacant positions after disqualifications (D8); the broken race times of the 2024 Belgian GP (Section 1.3); some cars that failed before the start are coded as 0-lap retirements and count as starts in Q3; and for raw grid 0, telling a pit-lane starter from a car that never started uses the post-race `laps` column (D6, D7), which affects a handful of rows.

How to reproduce

On Kaggle: attach the dataset, turn Internet on (the first cell installs `shap`, `lime` or `statsmodels` if missing), and press Run All. Locally: put the 14 CSVs in `data/`, install `requirements.txt`, run all cells. The notebook writes the tables and every figure of this report to `outputs/`; this report is rebuilt from `report/` with `python report/build.py`. The cleaning, the DE answers, the logistic regression and gradient boosting are deterministic; the random forest and the network are seeded and report their spread over three seeds.

References

- Ergast Developer API / Kaggle: Formula 1 World Championship (1950–2024), by R. Rao.
- van den Goorbergh, R., van Smeden, M., Timmerman, D., Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *JAMIA* 29(9), 1525–1534.

- Henderson, D. A., Kirrane, L. J. (2018). A comparison of truncated and time-weighted Plackett–Luce models for probabilistic forecasting of Formula One results. *Bayesian Analysis* 13(2), 335–358.
- Lundberg, S. M., Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS.
- Ribeiro, M. T., Singh, S., Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. KDD.
- Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *JASA* 22(158), 209–212.
- scikit-learn documentation (`average_precision_score`, `HistGradientBoostingClassifier`); Keras documentation.